

project_4_code_part_2

March 18, 2025

1 Question 9

```
[12]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[13]: import json

path = "/content/drive/MyDrive/Data Part 2/"
hashtags = ["gohawks", "gopatriots", "nfl", "patriots", "sb49", "superbowl"]

def count(file_path):
    c = 0
    with open(file_path, 'r') as f:
        for line in f:
            c+=1
    return c

hashtags = {h: {"file_name": "tweets_" + h + ".txt",
               "file_path": path + "tweets_" + h + ".txt",
               "count": count(path + "tweets_" + h + ".txt")
              }
            for h in hashtags}

attributes = ['firstpost_date', 'title', 'url', 'tweet', 'author',
             ↪ 'original_author', 'citation_date', 'metrics', 'highlight', 'type',
             ↪ 'citation_url']

for h in hashtags:
    print("#" + h + " "*(10-len(h)), (7-len(str(hashtags[h]["count"])))*" " +
    ↪ str(hashtags[h]["count"]), "tweets")
```

```
#gohawks      169122 tweets
#gopatriots   23511 tweets
#nfl          233022 tweets
#patriots     440621 tweets
#sb49         743649 tweets
```

```
#superbowl 1213813 tweets
```

```
[14]: import math
```

```
class TweetDataLoader:
    def __init__(self, hashtag, batch_size=0):
        self.hashtag = hashtag
        self.file_path = hashtags[self.hashtag]["file_path"]
        self.total_count = hashtags[self.hashtag]["count"]
        self.batch_size = batch_size
        if self.batch_size > 0:
            self._length = math.ceil(self.total_count / self.batch_size)
        else:
            self._length = self.total_count

        self.f = open(self.file_path, 'r', encoding='utf-8')
        self.offsets = []
        self._build_index()

    def _build_index(self):
        """Builds a list of byte offsets for each line in the file."""
        self.f.seek(0)
        pos = self.f.tell()
        line = self.f.readline()
        while line:
            self.offsets.append(pos)
            pos = self.f.tell()
            line = self.f.readline()

    def __len__(self):
        return self._length

    def __getitem__(self, index):
        if self.batch_size == 0:
            # Single-item mode.
            if index < 0:
                index = self.total_count + index
            if index < 0 or index >= self.total_count:
                raise IndexError("Index out of range")
            self.f.seek(self.offsets[index])
            line = self.f.readline()
            return json.loads(line)
        else:
            # Batch mode.
            if index < 0:
                index = self._length + index
            if index < 0 or index >= self._length:
```

```

        raise IndexError("Batch index out of range")
    start_index = index * self.batch_size
    end_index = min(start_index + self.batch_size, self.total_count)
    batch = []
    for i in range(start_index, end_index):
        self.f.seek(self.offsets[i])
        line = self.f.readline()
        batch.append(json.loads(line))
    return batch

def __del__(self):
    self.f.close()

```

```

[15]: for h in hashtags:
    exec(h+"Loader = TweetDataLoader('"+h+"')")
    exec("hashtags[h]['loader'] = "+h+"Loader")

```

```

[ ]: from datetime import datetime, timedelta

def compute_stats(loader):
    total_tweets = len(loader)
    total_followers = 0
    total_retweets = 0
    min_time = None
    max_time = None

    for i in range(total_tweets):
        tweet = loader[i]
        tweet_time = datetime.fromtimestamp(tweet['citation_date'])
        if min_time is None or tweet_time < min_time:
            min_time = tweet_time
        if max_time is None or tweet_time > max_time:
            max_time = tweet_time
        total_followers += tweet['author']['followers']
        total_retweets += tweet['metrics']['citations']['total']

    total_seconds = (max_time - min_time).total_seconds() if min_time and ↪
    max_time else 0
    total_hours = total_seconds / 3600.0 if total_seconds > 0 else 1

    avg_tweets_per_hour = total_tweets / total_hours
    avg_followers = total_followers / total_tweets
    avg_retweets = total_retweets / total_tweets

    return avg_tweets_per_hour, avg_followers, avg_retweets, min_time, max_time

stats_results = {}

```

```

for h in hashtags:
    print(f"Computing statistics for #{h}:")
    loader = hashtags[h]["loader"]
    avg_tph, avg_followers, avg_retweets, min_time, max_time = compute_stats(loader)

    stats_results[h] = {
        "tph": avg_tph,
        "fpt": avg_followers,
        "rpt": avg_retweets,
        "min_time": min_time.isoformat(),
        "max_time": max_time.isoformat(),
    }

    print(f" Average tweets per hour: {avg_tph:.2f}")
    print(f" Average followers per tweet: {avg_followers:.2f}")
    print(f" Average retweets per tweet: {avg_retweets:.2f}")
    time_window = max_time - min_time
    print(f" Min Time: {min_time.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f" Max Time: {max_time.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f" Time Window: {time_window}\n")

```

Computing statistics for #gohawks:

Average tweets per hour: 292.49
 Average followers per tweet: 2217.92
 Average retweets per tweet: 2.01
 Min Time: 2015-01-14 08:04:41
 Max Time: 2015-02-07 10:17:49
 Time Window: 24 days, 2:13:08

Computing statistics for #gopatriots:

Average tweets per hour: 40.95
 Average followers per tweet: 1427.25
 Average retweets per tweet: 1.41
 Min Time: 2015-01-14 09:50:11
 Max Time: 2015-02-07 07:54:35
 Time Window: 23 days, 22:04:24

Computing statistics for #nfl:

Average tweets per hour: 397.02
 Average followers per tweet: 4662.38
 Average retweets per tweet: 1.53
 Min Time: 2015-01-14 08:00:04
 Max Time: 2015-02-07 18:55:36
 Time Window: 24 days, 10:55:32

Computing statistics for #patriots:

Average tweets per hour: 750.89
Average followers per tweet: 3280.46
Average retweets per tweet: 1.79
Min Time: 2015-01-14 08:07:18
Max Time: 2015-02-07 18:55:00
Time Window: 24 days, 10:47:42

Computing statistics for #sb49:

Average tweets per hour: 1276.86
Average followers per tweet: 10374.16
Average retweets per tweet: 2.53
Min Time: 2015-01-14 12:31:15
Max Time: 2015-02-07 18:55:36
Time Window: 24 days, 6:24:21

Computing statistics for #superbowl:

Average tweets per hour: 2072.12
Average followers per tweet: 8814.97
Average retweets per tweet: 2.39
Min Time: 2015-01-14 08:13:07
Max Time: 2015-02-07 18:00:08
Time Window: 24 days, 9:47:01

```
[ ]: def compute_tweets_per_hour(loader, min_time, max_time):  
    current_bin = min_time.replace(minute=0, second=0, microsecond=0)  
    bins = []  
    while current_bin <= max_time:  
        bins.append(current_bin)  
        current_bin += timedelta(hours=1)  
    counts = {b: 0 for b in bins}  
    for i in range(loader.total_count):  
        tweet = loader[i]  
        tweet_time = datetime.fromtimestamp(tweet['citation_date'])  
        hour_bin = tweet_time.replace(minute=0, second=0, microsecond=0)  
        if hour_bin in counts:  
            counts[hour_bin] += 1  
        else:  
            counts[hour_bin] = 1  
  
    sorted_counts = sorted(counts.items())  
    df = pd.DataFrame(sorted_counts, columns=["Hour", "TweetCount"])  
    return df
```

```
[ ]: import pandas as pd  
import matplotlib.pyplot as plt
```

```

min_time_superbowl = datetime.
    ↳fromisoformat(stats_results["superbowl"]["min_time"])
max_time_superbowl = datetime.
    ↳fromisoformat(stats_results["superbowl"]["max_time"])
min_time_nfl = datetime.fromisoformat(stats_results["nfl"]["min_time"])
max_time_nfl = datetime.fromisoformat(stats_results["nfl"]["max_time"])

overall_min_time = min(min_time_superbowl, min_time_nfl)
overall_max_time = max(max_time_superbowl, max_time_nfl)

df_superbowl = compute_tweets_per_hour(superbowlLoader, overall_min_time,
    ↳overall_max_time)
df_nfl = compute_tweets_per_hour(nflLoader, overall_min_time, overall_max_time)

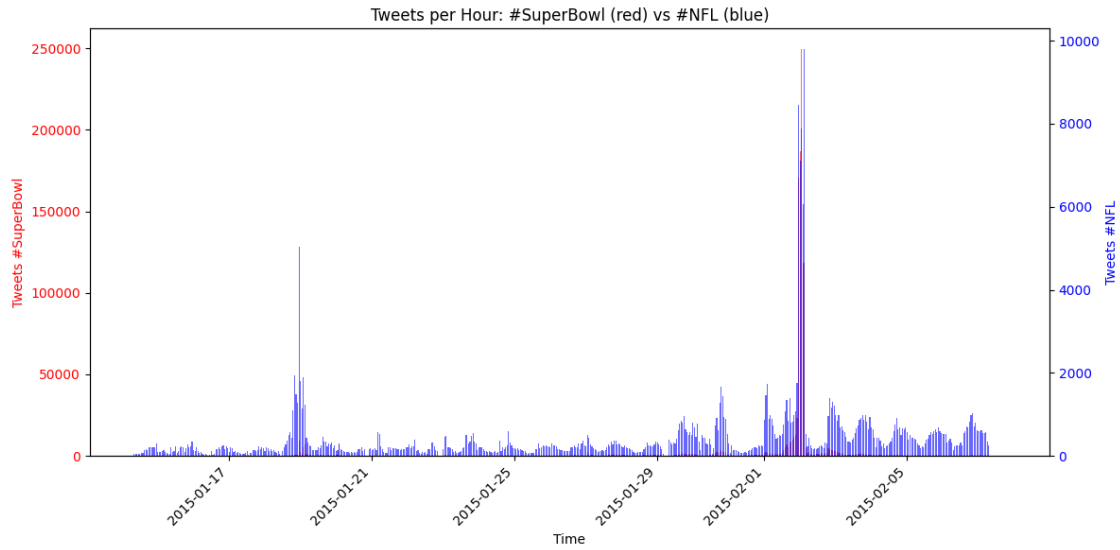
fig, ax1 = plt.subplots(figsize=(12, 6))
ax2 = ax1.twinx()

ax1.bar(df_superbowl["Hour"], df_superbowl["TweetCount"], width=0.03,
        color='red', alpha=0.6, label="#SuperBowl")
ax2.bar(df_nfl["Hour"], df_nfl["TweetCount"], width=0.03,
        color='blue', alpha=0.6, label="#NFL")

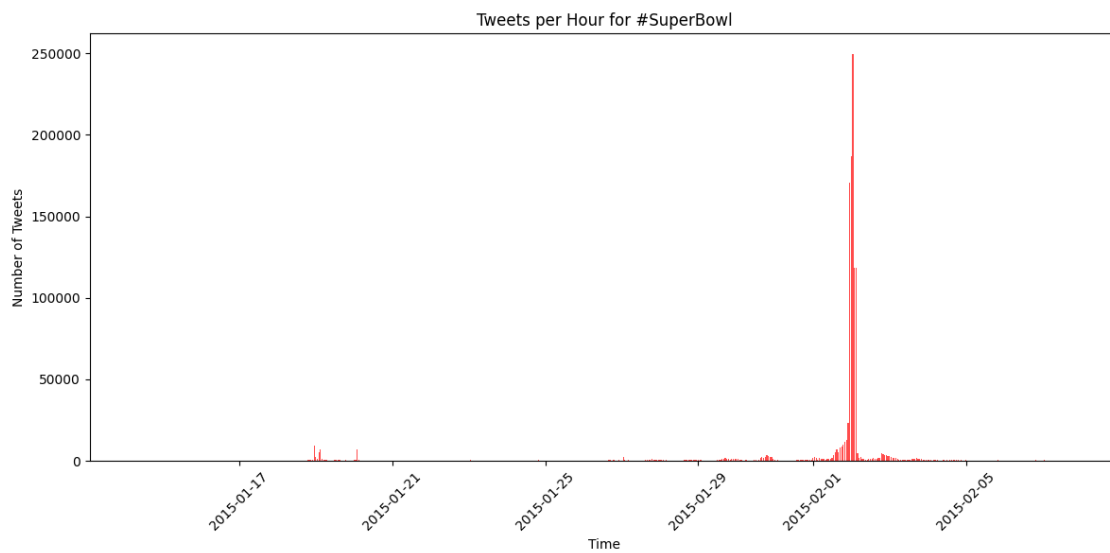
ax1.set_xlabel("Time")
ax1.set_ylabel("Tweets #SuperBowl", color='red')
ax2.set_ylabel("Tweets #NFL", color='blue')
ax1.tick_params(axis='y', labelcolor='red')
ax2.tick_params(axis='y', labelcolor='blue')
plt.title("Tweets per Hour: #SuperBowl (red) vs #NFL (blue)")

fig.autofmt_xdate(rotation=45)
plt.tight_layout()
plt.show()

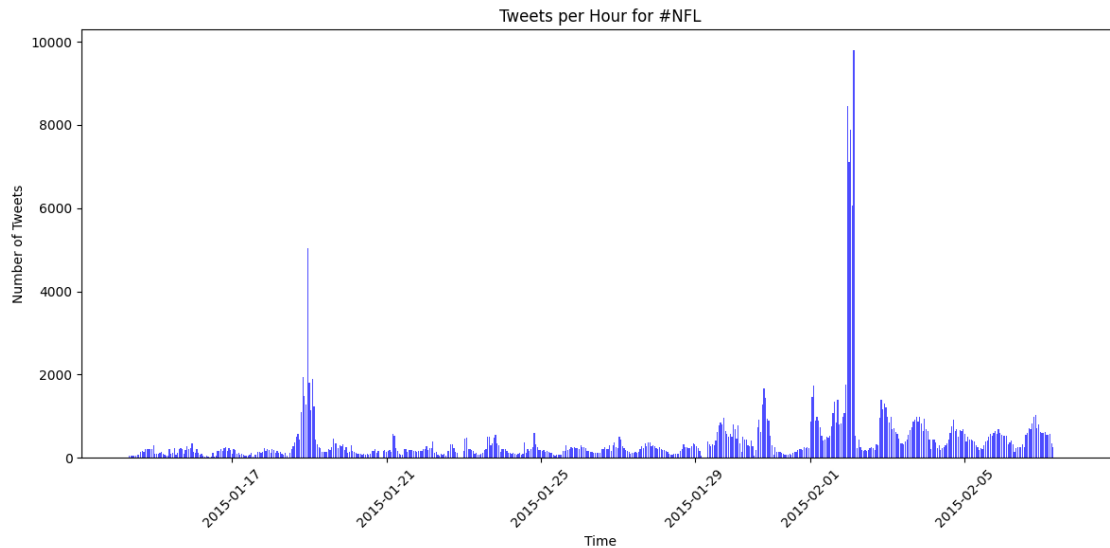
```



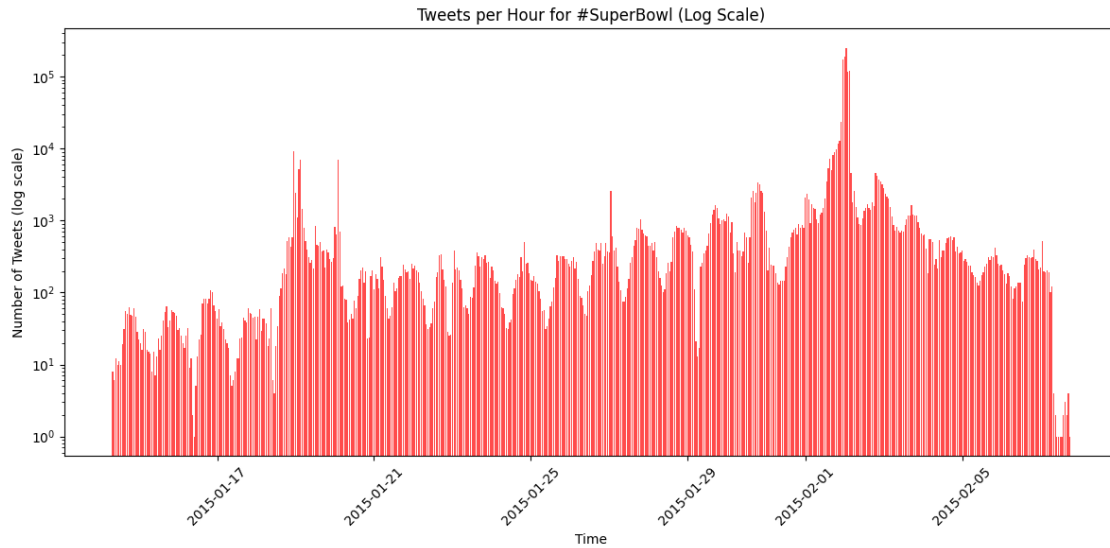
```
[ ]: plt.figure(figsize=(12,6))
plt.bar(df_superbowl["Hour"], df_superbowl["TweetCount"], width=0.03,
        color='red', alpha=0.7)
plt.title("Tweets per Hour for #SuperBowl")
plt.xlabel("Time")
plt.ylabel("Number of Tweets")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



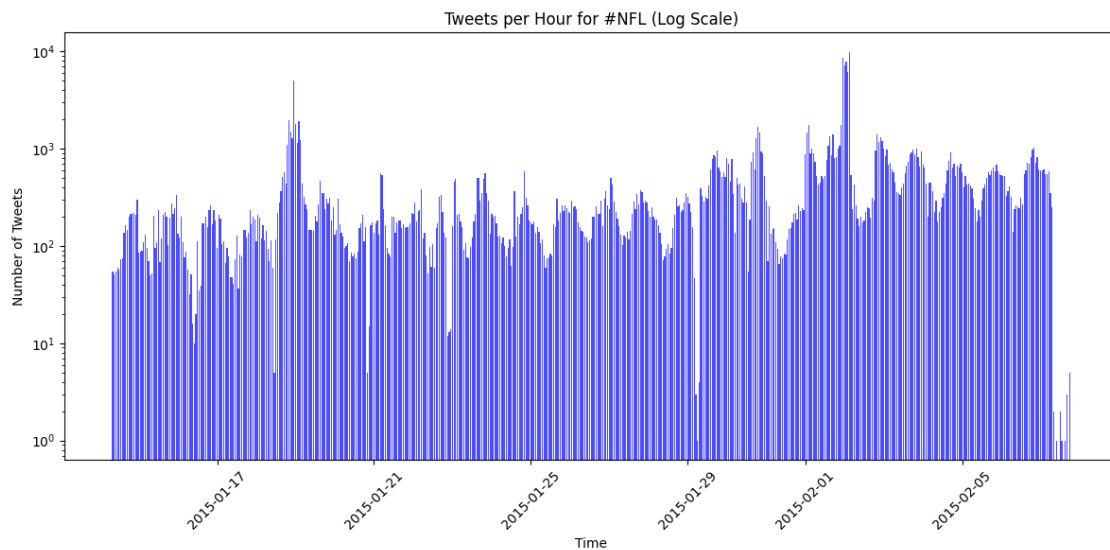
```
[ ]: plt.figure(figsize=(12,6))
plt.bar(df_nfl["Hour"], df_nfl["TweetCount"], width=0.03, color='blue', alpha=0.7)
plt.title("Tweets per Hour for #NFL")
plt.xlabel("Time")
plt.ylabel("Number of Tweets")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
[ ]: plt.figure(figsize=(12,6))
plt.bar(df_superbowl["Hour"], df_superbowl["TweetCount"], width=0.03, color='red', alpha=0.7)
plt.yscale("log")
plt.title("Tweets per Hour for #SuperBowl (Log Scale)")
plt.xlabel("Time")
plt.ylabel("Number of Tweets (log scale)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

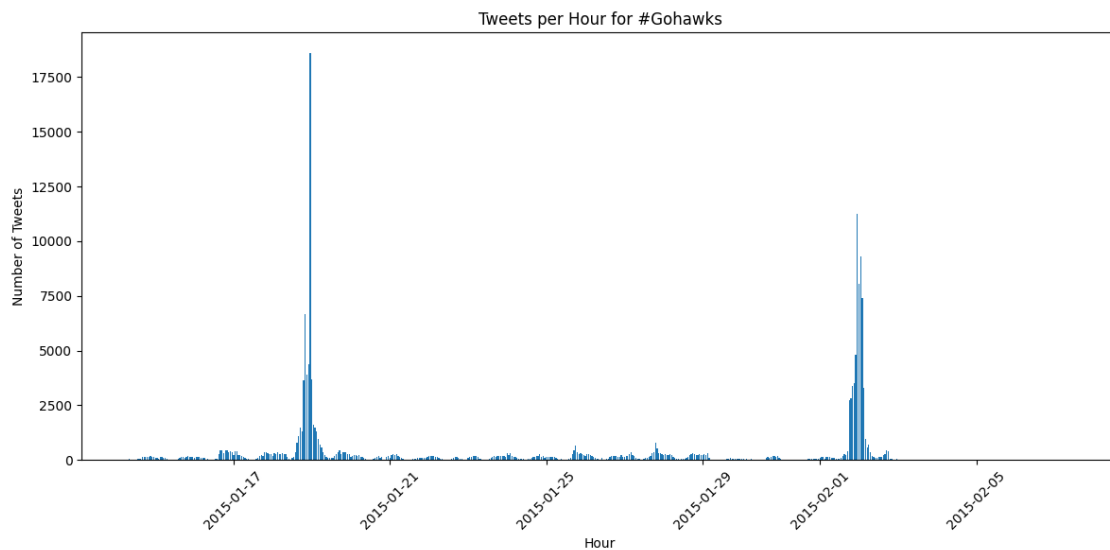
```
[ ]: plt.figure(figsize=(12,6))
plt.bar(df_nfl["Hour"], df_nfl["TweetCount"], width=0.03, color='blue', alpha=0.
↪7)
plt.yscale("log")
plt.title("Tweets per Hour for #NFL (Log Scale)")
plt.xlabel("Time")
plt.ylabel("Number of Tweets")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

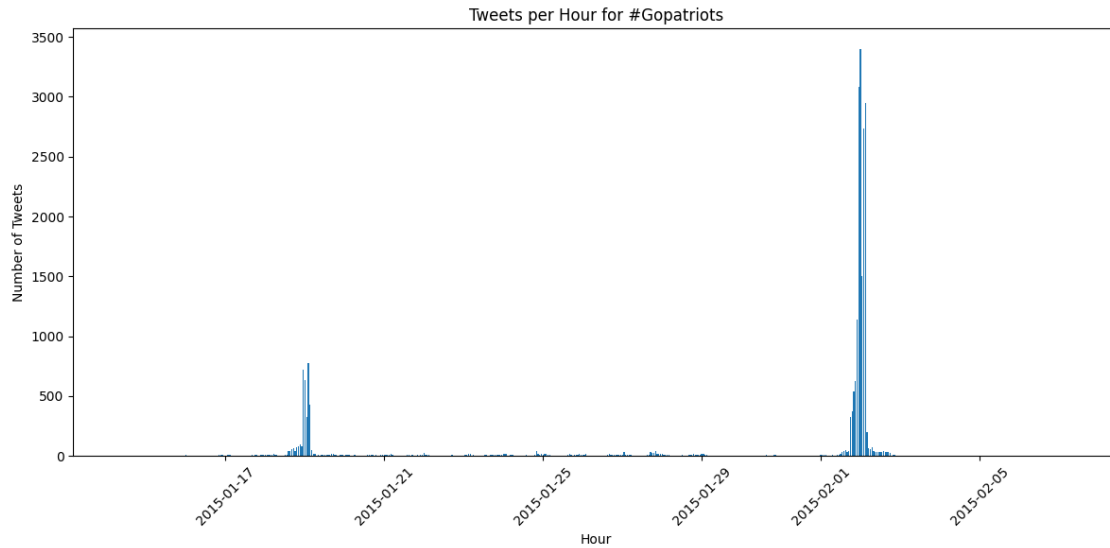


```
[ ]: for tag in ["gohawks", "gopatritots"]:
    # Get the loader and the previously computed stats.
    loader = hashtags[tag]["loader"]
    # Retrieve min_time and max_time from stats_results and convert them to
    ↪ datetime objects.
    min_time = datetime.fromisoformat(stats_results[tag]["min_time"])
    max_time = datetime.fromisoformat(stats_results[tag]["max_time"])

    # Compute the per-hour tweet counts using the established time window.
    df = compute_tweets_per_hour(loader, min_time, max_time)

    # Plot the bar chart.
    plt.figure(figsize=(12,6))
    plt.bar(df["Hour"], df["TweetCount"], width=0.03) # Adjust width if needed.
    plt.title(f"Tweets per Hour for #{tag.capitalize()}")
    plt.xlabel("Hour")
    plt.ylabel("Number of Tweets")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```





```
[ ]: df
```

```
[ ]:
      Hour  TweetCount
0  2015-01-14 09:00:00      1
1  2015-01-14 10:00:00      1
2  2015-01-14 11:00:00      0
3  2015-01-14 12:00:00      2
4  2015-01-14 13:00:00      1
..      ...
570 2015-02-07 03:00:00      0
571 2015-02-07 04:00:00      0
572 2015-02-07 05:00:00      0
573 2015-02-07 06:00:00      0
574 2015-02-07 07:00:00      1
```

```
[575 rows x 2 columns]
```

```
[ ]:
```

```
[ ]: import numpy as np

hours=5

peak=df_nfl["Hour"][np.argmax(df_nfl["TweetCount"])]
A=np.array([abs(a-peak)<=timedelta(hours=hours) for a in df_nfl["Hour"]])
print(round(np.sum(np.array(df_nfl["TweetCount"])[A])/np.
      ↳sum(df_nfl["TweetCount"])*100,1),"%")
```

```
18.3 %
```

2 Question 10

3 Predicting Tweet Impressions in a Sports Context

3.1 1. Data Preprocessing and Feature Engineering

- **Extract Relevant Tweets:**
 - Use a dataset containing tweets related to a specific sporting event, ensuring each tweet includes metadata such as timestamp, user information, and impression count (if available).
- **Data Cleaning:**
 - Clean tweet texts by removing URLs, special characters, and unnecessary artifacts (e.g., emojis, HTML tags).
- **Feature Engineering:**
 - Extract key features such as:
 - * **Tweet-based features:** Length, number of hashtags, mentions, media attachments.
 - * **User-based features:** Follower count, engagement history, verification status.
 - * **Time-based features:** Time of posting relative to game progress.

3.2 2. Sentiment and Topic Analysis

- **Sentiment Analysis:**
 - Apply sentiment analysis (e.g., VADER or a transformer-based model) to quantify the emotional tone of each tweet.
- **Topic Modeling:**
 - Use LDA or BERTopic to extract themes from the tweets and determine if certain topics correlate with higher engagement.
- **Lexical and Semantic Features:**
 - Extract NLP-based features, including TF-IDF, word embeddings, and phrase-level insights.

3.3 3. Temporal and Contextual Mapping

- **Timestamp Normalization:**
 - Convert timestamps to local event time, tracking how tweet timing affects impressions.
- **Game Event Overlays:**
 - Align tweets with in-game events (e.g., goals, touchdowns, substitutions) to explore their influence on tweet visibility.

3.4 4. Model Selection and Training

- **Data Splitting:**
 - Split the dataset into training, validation, and test sets based on chronological order to prevent data leakage.
- **Feature Selection and Baseline Models:**
 - Evaluate multiple models, including linear regression, decision trees, and random forests, for baseline comparisons.
- **Advanced Modeling:**
 - Use deep learning models (e.g., transformers or LSTMs) to capture sequential patterns in tweet engagement.
 - Experiment with ensemble methods for improved prediction accuracy.

3.5 5. Evaluation and Interpretation

- **Performance Metrics:**
 - Use metrics like Mean Absolute Error (MAE) and R^2 to evaluate prediction accuracy.
- **Error Analysis:**
 - Identify underperforming cases (e.g., tweets with unexpected high or low impressions).
- **Feature Importance Analysis:**
 - Use SHAP values or permutation importance to interpret which features contribute most to tweet impressions.

3.6 6. Application and Insights

- **Optimization Strategies:**
 - Recommend strategies for maximizing tweet reach based on insights from the model.
- **Comparative Analysis:**
 - Compare predicted impressions with actual engagement to validate the model's effectiveness in different scenarios.

```
[16]: !pip install langid
```

```
Requirement already satisfied: langid in /usr/local/lib/python3.11/dist-packages (1.1.6)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from langid) (1.26.4)
```

Let's get a feeling for our data.

```
[6]: import numpy as np

for i in range(15):
```

```
print(superbowlLoader[np.random.  
↳randint(0,len(superbowlLoader))]['tweet']['text'], "\n")
```

-- lets go patriots #SuperBowlXLIX #SuperBowlSunday #NINERGAANG

Missy Elliot??!!??? That was a throw back but a good show!! #SuperBowlXLIX

Your child wants 2b the next #SuperBowl star but u can't afford 2 send them 4 activities & u live in PA,NJ,DE look: <http://t.co/8Bf3MlrxBf>

Cuanta emoción ayer en #SuperBowlXLIX #NEvsSEA aqui los Highlights!!
#LoveUrShirtMan #PatriotsWIN #NFLTutorialMan <https://t.co/qCpN8mUgSz>

Sick of the predictions, sick of the talking...let's plsy some football!!!!
#SuperBowl49 #patriots

#throwback to michael jackson's 1993 #SuperBowl half-time show
<http://t.co/1lE75gjcz>

McDonalds are so full of shit. Haha. #SuperBowlXLIX

#SuperBowl Spin Cycle by Mixing & Remixing The Hits Spin Cycle
#iHeartRadio #NowPlaying <http://t.co/wjldHSEL7U> <http://t.co/4V4CJpv2KL>

Partidazo: NE Patriots VS Seattle Seahawks #SuperBowlXLIX #SuperBowl

Waiting for #TheBlacklist to come on, but they keep talking about the
#Superbowl. This is pretty much... <http://t.co/E8xUxcVwhZ>

I love seeing a @BoilerFootball alumni playing every defensive down during the
#SuperBowl #BoilerUp

What are with these sad #SuperBowlCommercials? #SuperBowlOfSadness

Super Bowl 49 it will be The @Seahawks vs The @Patriots @nfl #nfl #SuperBowl
#SuperBowlXLIX #ihatebothteams

Inicia el tercer cuarto. @Patriots 14-14 @Seahawks #SuperBowlXLIX
<http://t.co/bobA7MHRRq>

Most impressive thing about @katyperry was the wardrobe changes. More Lenny,
less Katy. #SuperBowl

These tweets are mainly one thing: dirty. Some consist mainly of emojis. Some are in spanish... we must filter for recognisable english containing mainly normal characters. We adapt the dataloader to filter.

```

[17]: import json
import math
import re
from datetime import datetime
import pytz
import langid

# Precompile an emoji pattern (covers common emoji ranges)
emoji_pattern = re.compile("[
    u"\U0001F600-\U0001F64F" # emoticons
    u"\U0001F300-\U0001F5FF" # symbols & pictographs
    u"\U0001F680-\U0001F6FF" # transport & map symbols
    u"\U0001F1E0-\U0001F1FF" # flags
    "]" + "", flags=re.UNICODE)

def is_valid_tweet(tweet, p=.3):
    """
    Returns True if tweet text is recognized as English,
    is not dominated by emojis (e.g. more than p=30% of characters are emoji),
    and is not composed mainly of hashtags (e.g. more than 70% of tokens are
    ↪hashtags).
    """
    # Extract text from tweet; adjust the key access if your tweet structure
    ↪differs.
    text = tweet.get('tweet', {}).get('text', '')
    if not text:
        return False

    # Use langid to detect language (langid.classify is very fast)
    lang, conf = langid.classify(text)
    if lang != 'en':
        return False

    # Check the ratio of emoji characters in the text.
    total_chars = len(text)
    if total_chars > 0:
        emoji_count = len(emoji_pattern.findall(text))
        if (emoji_count / total_chars) > p:
            return False

    # Split text into tokens and check the fraction that are hashtags.
    tokens = text.split()
    if tokens:
        hashtag_count = sum(1 for token in tokens if token.startswith('#'))
        if (hashtag_count / len(tokens)) > 0.7:
            return False

```

```

        return True

import pickle

def filter(tweet, new_start_ts=None, new_end_ts=None):
    citation_date = tweet.get('citation_date')
    if new_start_ts is not None and new_end_ts is not None:
        if citation_date is None or not (new_start_ts <= citation_date <=
↪new_end_ts):
            return False
    retweeted = tweet['tweet']['retweeted']
    if retweeted:
        return False

    return is_valid_tweet(tweet)

class FilteredDataLoader(TweetDataLoader):
    def __init__(self, hashtag, batch_size=0):
        self.hashtag = hashtag
        self.file_path = hashtags[self.hashtag]["file_path"]
        self.batch_size = batch_size

        mst_tz = pytz.timezone("MST")
        self.start_dt = datetime(2015, 2, 1, 16, 30, 0, tzinfo=mst_tz)
        self.end_dt = datetime(2015, 2, 2, 23, 59, 59, tzinfo=mst_tz)
        self.start_ts = self.start_dt.timestamp()
        self.end_ts = self.end_dt.timestamp()

        # Open the file and build the index.
        self.f = open(self.file_path, 'r', encoding='utf-8')
        self.offsets = []
        self._build_index()

    def _build_index(self):
        self.f.seek(0)
        pos = self.f.tell()
        line = self.f.readline()
        while line:
            try:
                tweet = json.loads(line)
            except Exception:
                pos = self.f.tell()
                line = self.f.readline()
                continue
            if filter(tweet):
                self.offsets.append(pos)
            pos = self.f.tell()

```



```

        line = self.f.readline()

    self.filtered_count = len(self.offsets)
    if self.batch_size > 0:
        self._length = math.ceil(self.filtered_count / self.batch_size)
    else:
        self._length = self.filtered_count

    def __len__(self):
        return self._length

    def __getitem__(self, index):
        if self.batch_size == 0:
            if index < 0:
                index = self.filtered_count + index
            if index < 0 or index >= self.filtered_count:
                raise IndexError("Index out of range")
            self.f.seek(self.offsets[index])
            line = self.f.readline()
            return json.loads(line)
        else:
            if index < 0:
                index = self._length + index
            if index < 0 or index >= self._length:
                raise IndexError("Batch index out of range")
            start_index = index * self.batch_size
            end_index = min(start_index + self.batch_size, self.filtered_count)
            batch = []
            for i in range(start_index, end_index):
                self.f.seek(self.offsets[i])
                line = self.f.readline()
                batch.append(json.loads(line))
            return batch

    def cut(self, filter, silent=True):
        new_offsets = []
        for pos in self.offsets:
            self.f.seek(pos)
            line = self.f.readline()
            try:
                tweet = json.loads(line)
                if filter(tweet):
                    new_offsets.append(pos)
            except Exception:
                continue # Skip corrupted lines
        self.offsets = new_offsets
        self.filtered_count = len(new_offsets)

```

```

        if self.batch_size > 0:
            self._length = math.ceil(len(new_offsets) / dataloader.batch_size)
        else:
            self._length = len(new_offsets)

        if not silent: print(self.hashtag+f"Loader now contains {self.
↳filtered_count} tweets.")

        return self

    def __del__(self):
        self.f.close()

    def __getstate__(self):
        state = self.__dict__.copy()
        if 'f' in state:
            del state['f']
        return state

    def __setstate__(self, state):
        self.__dict__.update(state)
        self.f = open(self.file_path, 'r', encoding='utf-8')

```

```

[ ]: for h in hashtags:
    print("Making cleaned Loader for #"+h)
    exec(h+"Loader = FilteredDataLoader('"+h+"')")
    exec("hashtags[h]['loader'] = "+h+"Loader")

```

```

Making cleaned Loader for #gohawks
Making cleaned Loader for #gopatriots
Making cleaned Loader for #nfl
Making cleaned Loader for #patriots
Making cleaned Loader for #sb49
Making cleaned Loader for #superbowl

```

```

[10]: for h in hashtags:
    loader=hashtags[h]["loader"]
    print(h)
    print(len(loader))
    print("")

```

```

gohawks
169122

```

```

gopatriots
23511

```

```

nfl

```

233022

patriots
440621

sb49
743649

superbowl
1213813

```
[ ]: for h in hashtags:  
    loader = hashtags[h]["loader"]  
    with open('/content/drive/MyDrive/pickles/'+h+'Loader.pkl', 'wb') as f_out:  
        pickle.dump(loader, f_out)
```

```
[18]: for h in hashtags:  
    with open('/content/drive/MyDrive/219_Final_project/pickles/'+h+'Loader.pkl',  
↳ 'rb') as f_in:  
        hashtags[h]['loader'] = pickle.load(f_in)  
        loader=hashtags[h]['loader']  
        exec(h+"Loader = loader")
```

```
[9]: import numpy as np  
    for i in range(15):  
        print(superbowlLoader[np.random.  
↳ randint(0,len(superbowlLoader))]['tweet']['text'], "\n")
```

There's one team we can all support #SuperBowl <http://t.co/PahPf0Ijfb>

@Seahawks & @Patriots good luck to both team in the #SuperBowl give us a good game!

She's a lion, a dark horse. What next??? #SuperBowlXLIX #HalftimeShowKatyPerry

How to get NFL swag - no credit card swiping required #SuperBowl

What ever happened to a good band playing good music? #SuperBowl Halftime show.
WTF was that, the opening scene to the Lion King?

Patriots- Pass to Vareem . #SeattleSeahawks #NewEnglandPatriots #football
#AmericanFootball #SuperBowl #SuperBowlXLIX #SB49

God bless the people on the east coast..I'm happy the #SuperBowl champs are
@Patriots So sorry the celebration is postponed by

Magical Interception but Sherman tho! #patriots #SB49 #SuperBowlXLIX

#RichardSherman #Seahawks #funny <https://t.co/OTVgn56YGZ>

Yay @katyperry is on! Now it's time to watch the #SuperBowl #Halftime
#SuperBowlXLIX

#SuperBowl #seahawks in trouble. Defence too banged up

Which one is your favorite? #SuperBowlCommercials | NBC Bay Area
<http://t.co/4vaGRHhWhM> via @nbcbayarea

I kinda miss the old hippy Lenny Kravitz #SuperBowl

Julian Edelman is white. Seriously. LOL #SuperBowlXLIX

This looks siiiiiick in person no deflated balls allowed #SuperBowlXLIX
<http://t.co/VKB7ut0Asf>

#SuperBowl 2015 Ticket Prices Most Expensive Ever - I4U News
<http://t.co/vIGhReDTNl> <http://t.co/CUtM2K6sbD>

```
[19]: for k in superbowlLoader[0]:  
      print(type(superbowlLoader[0][k])==dict)
```

False
False
False
True
True
True
False
True
False
False
False

```
[26]: tweet = superbowlLoader[0]  
  
print(tweet['tweet']['text'],  
      tweet['firstpost_date'],  
      tweet['tweet']['user']['verified'],  
      tweet['author']['name'],  
      tweet['metrics']['impressions'])
```

At <http://t.co/Vd0RW0eAed> -- #Seahawks #12thMAN #12 #SeahawkNation
#SuperBowlBound #Superbowl #Repeat #GoHawks ... <http://t.co/XSEFUKqEhN>
1419883838 False Becca Delgado 5

```
[27]: import pandas as pd
import pickle
from tqdm import tqdm
import csv

# Path to your pickles
pickle_dir = "/content/drive/MyDrive/219_Final_project/pickles/"
pickle_files = [
    "superbowlLoader.pkl",
    "sb49Loader.pkl",
    "patriotsLoader.pkl",
    "nflLoader.pkl",
    "gopatriotsLoader.pkl",
    "gohawksLoader.pkl"
]

# Flatten function (as before)
def flatten_json(tweet):
    return {
        'text': tweet['tweet']['text'],
        'time': tweet['firstpost_date'],
        'verified': tweet['tweet']['user']['verified'],
        'author': tweet['author']['name'],
        'impressions': tweet['metrics']['impressions']
    }

# Output CSV file
output_csv_path = "/content/drive/MyDrive/219_Final_project/all_tweets_flat.csv"

# Initialize CSV file with header
with open(output_csv_path, mode='w', newline='', encoding='utf-8') as f_out:
    writer = csv.DictWriter(f_out, fieldnames=['text', 'time', 'verified', 'author', 'impressions'])
    writer.writeheader()
```

```
[30]: # Now process each pickle and append tweets to the CSV
for file_name in pickle_files:
    print(f"Processing {file_name} ...")
    with open(pickle_dir + file_name, 'rb') as f_in:
        tweet_loader = pickle.load(f_in)

    # Write in chunks to avoid memory overload
    with open(output_csv_path, mode='a', newline='', encoding='utf-8') as f_out:
        writer = csv.DictWriter(f_out, fieldnames=['text', 'time', 'verified', 'author', 'impressions'])
        for tweet in tqdm(tweet_loader):
            flat_tweet = flatten_json(tweet)
```

```
writer.writerow(flat_tweet)
```

Processing superbowlLoader.pkl ...

1%| | 9326/918021 [00:00<01:20, 11251.71it/s]

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-30-d5df41106db7> in <cell line: 0>()
      8     with open(output_csv_path, mode='a', newline='', encoding='utf-8')
->as f_out:
      9         writer = csv.DictWriter(f_out, fieldnames=['text', 'time',
->'verified', 'author', 'impressions'])
---> 10     for tweet in tqdm(tweet_loader):
      11         flat_tweet = flatten_json(tweet)
      12         writer.writerow(flat_tweet)

/usr/local/lib/python3.11/dist-packages/tqdm/std.py in __iter__(self)
    1179
    1180     try:
-> 1181         for obj in iterable:
    1182             yield obj
    1183             # Update and possibly print the progressbar.

<ipython-input-17-6c529eaad942> in __getitem__(self, index)
    108         raise IndexError("Index out of range")
    109         self.f.seek(self.offsets[index])
--> 110         line = self.f.readline()
    111         return json.loads(line)
    112     else:

/usr/lib/python3.11/codecs.py in decode(self, input, final)

KeyboardInterrupt:
```

```
[31]: for i in range(1000):
      try: superbowlLoader[i]['firstpost_date']
      except: print(i)
```

4 Feature engineering

```
[1]: # Import everything
import torchvision
!pip install ethnicolr
import ethnicolr
import torchvision
```

```

import os
import re
import json
import math
import random
import pickle
import numpy as np
import pandas as pd
from datetime import datetime
import pytz

import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
nltk.download('vader_lexicon')

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# Set random seeds for reproducibility
np.random.seed(42)
random.seed(42)

```

Requirement already satisfied: ethnicolr in /usr/local/lib/python3.11/dist-packages (0.9.8)

Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/dist-packages (from ethnicolr) (2.2.3)

Requirement already satisfied: tensorflow<2.16,>=2.7.2 in /usr/local/lib/python3.11/dist-packages (from ethnicolr) (2.15.1)

Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (1.26.4)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2025.1)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2025.1)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (2.1.0)

Requirement already satisfied: astunparse>=1.6.0 in

/usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (1.6.3)
 Requirement already satisfied: flatbuffers>=23.5.26 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (25.2.10)
 Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (0.6.0)
 Requirement already satisfied: google-pasta>=0.1.1 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (0.2.0)
 Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (3.13.0)
 Requirement already satisfied: libclang>=13.0.0 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (18.1.1)
 Requirement already satisfied: ml-dtypes~=0.3.1 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (0.3.2)
 Requirement already satisfied: opt-einsum>=2.3.2 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (3.4.0)
 Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-
 packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (24.2)
 Requirement already satisfied:
 protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3
 in /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (4.25.6)
 Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-
 packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (76.1.0)
 Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-
 packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (1.17.0)
 Requirement already satisfied: termcolor>=1.1.0 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (2.5.0)
 Requirement already satisfied: typing-extensions>=3.6.6 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (4.12.2)
 Requirement already satisfied: wrapt<1.15,>=1.11.0 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (1.14.1)
 Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (0.37.1)
 Requirement already satisfied: grpcio<2.0,>=1.24.3 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorflow<2.16,>=2.7.2->ethnicolr) (1.71.0)
 Requirement already satisfied: tensorboard<2.16,>=2.15 in


```

/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.2)
Requirement already satisfied: tensorflow-estimator<2.16,>=2.15.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.0)
Requirement already satisfied: keras<2.16,>=2.15.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.11/dist-packages (from
astunparse>=1.6.0->tensorflow<2.16,>=2.7.2->ethnicolr) (0.45.1)
Requirement already satisfied: google-auth<3,>=1.6.3 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (2.38.0)
Requirement already satisfied: google-auth-oauthlib<2,>=0.5 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (1.2.1)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.7)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (2.32.3)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from
tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.1.3)
Requirement already satisfied: cachetools<6.0,>=2.0.0 in
/usr/local/lib/python3.11/dist-packages (from google-
auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
(5.5.2)
Requirement already satisfied: pyasn1-modules>=0.2.1 in
/usr/local/lib/python3.11/dist-packages (from google-
auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
(0.4.1)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.11/dist-
packages (from google-
auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
(4.9)
Requirement already satisfied: requests-oauthlib>=0.7.0 in
/usr/local/lib/python3.11/dist-packages (from google-auth-
oauthlib<2,>=0.5->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
(2.0.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0-
>tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.4.1)

```

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (2025.1.31)

Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.0.2)

Requirement already satisfied: pyasn1<0.7.0,>=0.4.6 in /usr/local/lib/python3.11/dist-packages (from pyasn1-modules>=0.2.1->google-auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (0.6.1)

Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from requests-oauthlib>=0.7.0->google-auth-oauthlib<2,>=0.5->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.2.2)

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...

```
[32]: def clean_text(text):
        """
        Clean tweet text by removing URLs, HTML tags, non-ASCII characters,
        and extra whitespace.
        """
        # Remove URLs
        text = re.sub(r'https?:\/\/\S+', '', text)
        # Remove HTML tags
        text = re.sub(r'<.*?>', '', text)
        # Remove non-ASCII characters (this may remove emojis)
        text = text.encode('ascii', 'ignore').decode('ascii')
        # Remove extra whitespace
        text = re.sub(r'\s+', ' ', text).strip()
        return text
```

For demographic information from the name we will use a pretrained model, namely the one developed [here](#):

Predicting Twitter User Demographics from Names Alone Zach Wood-Doughty, Nicholas Andrews, Rebecca Marvin, Mark Dredze Center for Language and Speech Processing Johns Hopkins University, Baltimore, MD 21218

```
[ ]:
```

```
[3]: from transformers import pipeline
```

```
model_name = "padmajabfrl/Gender-Classification"  
classifier = pipeline("text-classification", model=model_name,  
    ↳trust_remote_code=True)  
print(classifier("Hello world!"))
```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94:

UserWarning:

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(

config.json: 0%| | 0.00/723 [00:00<?, ?B/s]

pytorch_model.bin: 0%| | 0.00/268M [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/360 [00:00<?, ?B/s]

vocab.txt: 0%| | 0.00/232k [00:00<?, ?B/s]

tokenizer.json: 0%| | 0.00/712k [00:00<?, ?B/s]

special_tokens_map.json: 0%| | 0.00/125 [00:00<?, ?B/s]

Device set to use cuda:0

[{'label': 'Female', 'score': 0.9857402443885803}]

```
[4]: import pandas as pd  
import numpy as np  
import ethnicolr as ec  
from transformers import pipeline
```

```
# Initialize the gender classifier pipeline.
```

```
gender_classifier = pipeline(  
    "text-classification",  
    model="padmajabfrl/Gender-Classification",  
    tokenizer="padmajabfrl/Gender-Classification"  
)
```

```
def splitname(s: str) -> tuple[str, str]:
```

```
    """
```

```
    Splits name into first and last.
```

```
    """
```

```

s = s.strip()
idx = s.rfind(' ')
if idx == -1:
    return ("", s) # No spaces found
return (s[:idx], s[idx+1:])

def get_race_prediction(full_name: str) -> str:
    """
    Predicts race from a full name using ethnicolr's Florida model.
    Returns the predicted race as a string.
    """
    df_temp = pd.DataFrame({"name": [full_name]})
    df_temp[['fname', 'lname']] = df_temp['name'].apply(lambda x: pd.
↳Series(splitname(x)))
    df_temp = ec.pred_fl_reg_name(df_temp, "fname", "lname")
    # Ethnicolr's function typically adds a 'race' column with the prediction.
    return df_temp.loc[0, "race"]

def get_gender_prediction(full_name: str) -> str:
    """
    Predicts gender from a full name using the padmajabfrl/
↳Gender-Classification model.
    Returns the predicted gender (e.g., 'male' or 'female') as a string.
    """
    result = gender_classifier(full_name)
    return result[0]['label'].lower()

# Define fixed dense vectors for each predicted race.
race_to_vector = {
    "white": np.eye(1,5,0).squeeze(),
    "black": np.eye(1,5,1).squeeze(),
    "asian": np.eye(1,5,2).squeeze(),
    "hispanic": np.eye(1,5,3).squeeze(),
    "other": np.eye(1,5,4).squeeze()}

# Define fixed dense vectors for each predicted gender.
gender_to_vector = {
    "male": np.array([1]),
    "female": np.array([-1]),
    "unknown": np.array([0])
}

def get_demographic_embedding(full_name: str) -> np.ndarray:
    """
    Given a full name, uses ethnicolr to predict race and a pretrained gender_
↳classifier to predict gender.

```

Then, maps the race and gender predictions to fixed dense vectors and concatenates them into a single vector.

Returns:

A numpy array representing the combined demographic embedding.

"""

```
race = get_race_prediction(full_name)
gender = get_gender_prediction(full_name)
```

Map predictions to dense vectors. Use "other" and "unknown" as defaults.

```
race_vec = race_to_vector.get(race.lower(), race_to_vector["other"])
```

```
gender_vec = gender_to_vector.get(gender.lower(),
```

```
gender_to_vector["unknown"])
```

Concatenate the race vector and the gender vector.

```
combined_embedding = np.concatenate([race_vec, gender_vec])
```

```
return combined_embedding
```

Example usage:

```
if __name__ == "__main__":
```

```
    sample_name = "Kim JongUn"
```

```
    demo_embedding = get_demographic_embedding(sample_name)
```

```
    print("Combined Demographic Embedding for", sample_name)
```

```
    print("Shape:", demo_embedding.shape)
```

```
    print("Embedding vector:", demo_embedding)
```

```
model.safetensors: 0%|          | 0.00/268M [00:00<?, ?B/s]
```

```
Device set to use cuda:0
```

```
Combined Demographic Embedding for Kim JongUn
```

```
Shape: (6,)
```

```
Embedding vector: [0. 0. 1. 0. 0. 1.]
```

[5]: # Load a pretrained SentenceTransformer for tweet text representation.

```
from sentence_transformers import SentenceTransformer
```

```
tweet_text_model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
def get_tweet_text_embedding(text: str) -> np.ndarray:
```

"""

Returns a dense vector representation of the input tweet text using a

pretrained model.

"""

```
    cleaned = clean_text(text)
```

```
    return tweet_text_model.encode(cleaned)
```

```
modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
```

```
config_sentence_transformers.json: 0%|          | 0.00/116 [00:00<?, ?B/s]
```

```

README.md: 0%|          | 0.00/10.5k [00:00<?, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/612 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/90.9M [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/350 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]

```

```

[6]: def get_time_representation(timestamp: float) -> np.ndarray:
    """
    Given a UNIX timestamp, returns a 4-dimensional cyclical time_
    ↪representation:
        [sin(2 * hour / 24), cos(2 * hour / 24), sin(2 * day / 7), cos(2 * day / 7)]
    """
    dt = datetime.fromtimestamp(timestamp)
    minute = dt.minute
    hour = dt.hour
    day = dt.weekday() # Monday = 0, Sunday = 6
    sin_minute = np.sin(2 * np.pi * minute / 60)
    cos_minute = np.cos(2 * np.pi * minute / 60)
    sin_hour = np.sin(2 * np.pi * hour / 24)
    cos_hour = np.cos(2 * np.pi * hour / 24)
    sin_day = np.sin(2 * np.pi * day / 7)
    cos_day = np.cos(2 * np.pi * day / 7)
    return np.array([sin_minute, cos_minute, sin_hour, cos_hour, sin_day,
    ↪cos_day])

def get_date_representation(timestamp: float) -> np.ndarray:
    """
    Given a UNIX timestamp, returns a 1-dimensional vector representing the_
    ↪absolute date.
    Here we convert the date to an ordinal (number of days since (2015, 2, 1)).
    """
    dt = datetime.fromtimestamp(timestamp)
    ref_date = datetime(2015, 2, 1)
    ordinal = dt.toordinal() - ref_date.toordinal()
    return np.array([ordinal])

```

```

[7]: import numpy as np
from datetime import datetime
from sentence_transformers import SentenceTransformer

```

```

from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Initialize VADER sentiment analyzer.
sentiment_analyzer = SentimentIntensityAnalyzer()

def get_combined_representation(tweet: dict) -> np.ndarray:
    """
    Given a tweet dictionary with keys:
        - 'tweet_text': raw tweet text.
        - 'timestamp': tweet timestamp (in seconds).
        - 'user_name': user's full name.
        - 'verified': boolean indicating if user is verified.

    Returns a vector by concatenating:
        1. Tweet text embedding.
        2. Cyclical time representation (4 dims).
        3. Absolute date representation (1 dim).
        4. Demographic embedding from the user's name.
        5. Sentiment representation (1 dim).
        6. Verified status (1 dim, as float 1.0 or 0.0).
    """
    # Tweet text embedding.
    text = tweet.get('tweet_text', '')
    text_emb = get_tweet_text_embedding(text)

    # Time and date representations.
    ts = tweet.get('timestamp', None)
    if ts is None:
        time_emb = np.zeros(4)
        date_emb = np.zeros(1)
    else:
        ts = float(ts)
        time_emb = get_time_representation(ts)
        date_emb = get_date_representation(ts)

    # Demographic embedding.
    name = tweet.get('user_name', '')
    demo_emb = get_demographic_embedding(name)

    # Sentiment representation.
    cleaned = clean_text(text)
    sentiment = sentiment_analyzer.polarity_scores(cleaned)['compound']
    sentiment_emb = np.array([sentiment])

    # Verified status as float.
    verified_flag = 1.0 if tweet.get('verified', False) else -1.0
    verified_emb = np.array([verified_flag])

```

```

# Concatenate all features into one vector.
combined_vector = np.concatenate([text_emb, time_emb, date_emb, demo_emb,
    sentiment_emb, verified_emb])
return combined_vector

```

```

[8]: tweet={"tweet_text" : "I absolutely hate this game and especially that dickhead,
    tom brady!",
    'timestamp': 1419883838,
    'user_name': "yuanyi yang",
    'verified': True
}

```

```

[9]: get_combined_representation(tweet).shape

```

```

[9]: (399,)

```

```

[10]: print(get_combined_representation(tweet))

```

```

[-3.83004472e-02  7.31776655e-02  1.82008091e-02 -1.87365100e-01
  6.81769103e-02 -1.86250266e-02 -3.20706740e-02  3.74338254e-02
  8.39423165e-02  5.10790236e-02 -8.38662758e-02 -1.26404418e-02
  8.54208767e-02 -1.52010368e-02 -1.02417305e-01 -9.93942004e-03
 -4.36289832e-02  1.68777164e-02  7.42347986e-02  8.91406741e-03
 -6.49267584e-02  1.03673622e-01 -8.18080083e-02 -9.44703165e-03
 -5.02187312e-02 -1.11356908e-02  5.16900010e-02  2.24986561e-02
 -1.00287519e-01 -8.36424008e-02  8.96348432e-02 -3.10084373e-02
 -1.26406401e-02 -2.88689267e-02 -9.17212293e-02  4.66750516e-03
  1.46747855e-02 -1.92168187e-02  5.29285260e-02  5.29416725e-02
 -3.50605473e-02 -2.28044167e-02  5.93337417e-02  3.62425782e-02
 -2.79110856e-03  3.49369453e-04 -5.75162843e-03 -7.01493621e-02
  1.23996146e-01 -2.56053768e-02  5.87602258e-02  4.33790758e-02
  8.24870914e-03 -1.56401545e-02  3.57825719e-02  3.56494039e-02
  5.16157597e-02  5.48866689e-02 -5.93955331e-02  1.99110787e-02
 -4.83915918e-02 -3.59567255e-02 -7.20321946e-03 -4.36701812e-02
  2.23082453e-02  9.30478424e-03  6.08249428e-03  5.91432035e-04
 -3.73402908e-02  5.09857312e-02 -4.68491875e-02  7.74629265e-02
  2.95078028e-02  1.74150262e-02  4.80061118e-03 -1.84584670e-02
 -9.65077579e-02  3.00035230e-03  4.65442240e-02  2.69231908e-02
 -2.62304978e-03 -6.15702532e-02 -5.00220656e-02  6.52723759e-02
  3.43331061e-02  2.00743657e-02 -2.45785043e-02  6.64859489e-02
  3.72227356e-02  1.41098641e-03 -3.84532064e-02 -1.07679078e-02
  2.72740666e-02  1.09017557e-02  1.89988669e-02 -1.43483058e-02
 -2.23049112e-02 -5.86897843e-02 -1.35088250e-01  5.78044765e-02
 -2.60891952e-02  4.37002908e-03 -5.68981096e-02 -7.54798343e-03
  1.08713865e-01  5.42811044e-02 -4.88212099e-03  9.47871711e-03
 -4.13673073e-02 -5.99788129e-02  5.87073108e-03  3.43249068e-02
 -1.00462459e-01 -2.03856193e-02 -3.43476348e-02  3.72294895e-02

```


6.93884045e-02	2.27930807e-02	-4.33340296e-02	-2.46192534e-02
8.09193999e-02	8.58563632e-02	1.95056424e-02	4.40647490e-02
-7.09256111e-03	-5.72241619e-02	-1.89036336e-02	-3.33655219e-33
-1.49751091e-02	1.58694573e-02	-2.12110654e-02	-2.91110836e-02
-1.65531747e-02	9.43581760e-02	2.37796996e-02	3.96587141e-02
-3.07747163e-02	-2.61493213e-02	2.59294640e-02	-2.04136074e-02
-1.05352029e-01	-3.36039555e-03	6.16561435e-03	-7.94740990e-02
-4.13738973e-02	-1.95830390e-02	-8.47880077e-03	-1.57604907e-02
-3.81888752e-03	4.13311645e-02	4.27874224e-03	5.14621213e-02
-7.61708915e-02	2.74065640e-02	-1.51878633e-02	-8.07873383e-02
-7.62985880e-03	-5.00590866e-03	-4.59410474e-02	-2.83461343e-02
-5.90536334e-02	6.60866126e-02	2.81407665e-02	-1.36321932e-01
-5.42641431e-02	6.86525404e-02	5.15679903e-02	2.43953597e-02
-2.16818210e-02	-3.42993699e-02	-1.02129942e-02	6.57115597e-04
-1.91227775e-02	-4.87229042e-03	1.16234221e-01	-4.17776108e-02
7.33372867e-02	-3.44217233e-02	6.24084808e-02	-2.88314875e-02
1.03888847e-01	5.72620779e-02	2.81817485e-02	-1.07360877e-01
3.19157243e-02	-1.02117486e-01	-6.19401522e-02	1.80864017e-02
6.20226935e-03	3.78831737e-02	4.17817682e-02	-2.96025183e-02
-8.10201168e-02	-2.44131498e-03	-5.04285730e-02	2.55868919e-02
-2.40758434e-03	-2.77037099e-02	8.29412118e-02	-2.80883871e-02
1.17975818e-02	-7.20043555e-02	6.95324037e-03	-2.32643019e-02
9.11173522e-02	6.99754059e-02	-5.73840402e-02	-1.29549816e-01
6.51035234e-02	-4.24783267e-02	1.80272758e-02	-1.18333183e-01
2.11733691e-02	2.12667510e-02	3.87196280e-02	-1.06151864e-01
8.35893024e-03	-1.01748044e-02	1.06356367e-02	-1.63085666e-02
-6.61421428e-03	-1.00723160e-02	-1.55054480e-02	7.65879661e-34
-1.26674713e-03	-7.60569647e-02	4.68149781e-02	-7.60409189e-03
-5.79542667e-02	-7.57835656e-02	2.01281011e-02	6.40799254e-02
4.83431434e-03	1.25270495e-02	-1.44652370e-02	5.46971336e-02
-5.04316725e-02	-2.99887657e-02	1.84304100e-02	-3.00629083e-02
2.86035798e-02	-1.59327183e-02	8.37368704e-03	-7.47558996e-02
4.77315634e-02	2.04654709e-02	-4.98013347e-02	-1.54062165e-02
-5.84829133e-03	1.02433478e-02	-4.25889157e-02	2.22138558e-02
4.47267331e-02	-3.67738791e-02	3.32012177e-02	6.45059198e-02
-4.04424313e-03	-2.08599512e-02	-1.73668079e-02	8.73154104e-02
2.13774797e-02	1.30568773e-01	1.04960296e-02	-5.00828996e-02
2.43397951e-02	-9.27230567e-02	-1.42297670e-02	3.68505307e-02
1.82716046e-02	3.90518047e-02	-6.88917469e-04	-3.41673233e-02
-8.15836340e-02	1.01294123e-01	-1.25608400e-01	7.20814019e-02
2.33555050e-03	-3.03259194e-02	-7.39604011e-02	-1.73661494e-04
2.44402513e-02	3.40147577e-02	4.87813614e-02	-1.13617210e-02
-8.53547920e-03	-3.42616066e-02	-1.54126853e-01	5.28398231e-02
1.12621315e-01	3.93785872e-02	2.17113830e-03	-6.72838390e-02
-4.23540212e-02	1.17462024e-01	-1.14029087e-01	-5.91790788e-02
-8.97823870e-02	-2.62620784e-02	4.11708653e-02	2.82396097e-02
-3.00120711e-02	5.67665622e-02	-3.06363646e-02	1.08922213e-01
-1.03895348e-02	4.30963002e-02	1.92665949e-03	3.36778760e-02

```

-5.07099070e-02  4.75738794e-02  2.38748454e-03  7.10158981e-03
-7.53048062e-02  3.68449576e-02 -1.20116184e-02  1.10997595e-02
-3.22039500e-02  2.32731700e-02  4.33301218e-02 -1.93280858e-08
-4.87918407e-02 -2.89688464e-02 -1.33075230e-02 -5.57878762e-02
 6.63301628e-03 -4.12840657e-02 -7.70058930e-02  6.72653988e-02
 7.18577728e-02  4.17905338e-02  6.31462829e-03  2.80039869e-02
-4.71601151e-02  2.90190168e-02  1.02435681e-03  9.65621769e-02
 5.50774038e-02 -1.18024116e-02  1.14315683e-02  2.21448615e-02
 8.05435330e-03  5.18008284e-02 -6.68404996e-02 -3.83744650e-02
-6.73449319e-03 -4.73591574e-02  8.50979891e-03  1.22143179e-02
 7.95731843e-02  2.15644520e-02  3.98043245e-02 -1.55950589e-02
-3.50196660e-02 -3.88538018e-02 -5.06610088e-02  9.79106128e-03
 8.56731646e-03  1.02271568e-02  3.47472243e-02 -4.63850200e-02
 1.79181993e-02  4.36829925e-02 -8.97062123e-02 -2.22118385e-02
 4.27232264e-03  1.16117951e-02  6.01047277e-03  2.75055580e-02
 7.86708016e-03 -1.21836225e-02  4.21324074e-02  8.57621357e-02
-2.04917658e-02  4.84324805e-02  9.27668661e-02 -4.94849264e-05
 3.94690456e-03  9.86809097e-03  3.14675868e-02  2.65352484e-02
 3.63173857e-02  8.77794549e-02  3.63664478e-02  2.24804506e-02
 8.66025404e-01  5.00000000e-01 -8.66025404e-01  5.00000000e-01
 0.00000000e+00  1.00000000e+00 -3.40000000e+01  0.00000000e+00
 0.00000000e+00  1.00000000e+00  0.00000000e+00  0.00000000e+00
-1.00000000e+00 -8.64600000e-01  1.00000000e+00]

```

```

[33]: df = pd.read_csv(output_csv_path)
      print(df.head())

```

		text	time	verified	\
0	At	http://t.co/Vd0RW0eAed -- #Seahawks #12thMA...	1419883838	False	
1	You been 12ed	pass it on #SeahawkNation #LOB #...	1420343180	False	
2	27 days to the SuperBowl	\n#Katyperry #KatyC...	1420354386	False	
3	Check out the cool event that	#budlight has p...	1420662396	False	
4	Our @ButchStearns talked	#Patriots w/3X #Super...	1421030046	True	

	author	impressions
0	Becca Delgado	5
1	Becca Delgado	4
2	Walterlinesjr	833
3	EventMarketer	13943
4	Coach Chrissy Carew	5580

```

[34]: # Check for GPU (T4)
import torch
print(torch.cuda.get_device_name(0))

# Install dependencies
!pip install sentence-transformers ethnicolr transformers nltk

```

```
import nltk
nltk.download('vader_lexicon')
```

Tesla T4

Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.11/dist-packages (3.4.1)
Requirement already satisfied: ethnicolr in /usr/local/lib/python3.11/dist-packages (0.9.8)
Requirement already satisfied: transformers in /usr/local/lib/python3.11/dist-packages (4.48.3)
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (4.67.1)
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (2.6.0+cu124)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (1.6.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (1.14.1)
Requirement already satisfied: huggingface-hub>=0.20.0 in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (0.28.1)
Requirement already satisfied: Pillow in /usr/local/lib/python3.11/dist-packages (from sentence-transformers) (11.1.0)
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.11/dist-packages (from ethnicolr) (2.2.3)
Requirement already satisfied: tensorflow<2.16,>=2.7.2 in /usr/local/lib/python3.11/dist-packages (from ethnicolr) (2.15.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from transformers) (3.17.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from transformers) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.11/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: tokenizers<0.22,>=0.21 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.21.1)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.11/dist-packages (from transformers) (0.5.3)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)

Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (2024.10.0)

Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.20.0->sentence-transformers) (4.12.2)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2025.1)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.3.0->ethnicolr) (2025.1)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (2.1.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (1.6.3)

Requirement already satisfied: flatbuffers>=23.5.26 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (25.2.10)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (0.6.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (0.2.0)

Requirement already satisfied: h5py>=2.9.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (3.13.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (18.1.1)

Requirement already satisfied: ml-dtypes~=0.3.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (0.3.2)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (3.4.0)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (4.25.6)

Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (76.1.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.16,>=2.7.2->ethnicolr) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from

```

tensorflow<2.16,>=2.7.2->ethnicolr) (2.5.0)
Requirement already satisfied: wrapt<1.15,>=1.11.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (1.14.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (0.37.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (1.71.0)
Requirement already satisfied: tensorboard<2.16,>=2.15 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.2)
Requirement already satisfied: tensorflow-estimator<2.16,>=2.15.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.0)
Requirement already satisfied: keras<2.16,>=2.15.0 in
/usr/local/lib/python3.11/dist-packages (from
tensorflow<2.16,>=2.7.2->ethnicolr) (2.15.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-
packages (from torch>=1.11.0->sentence-transformers) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages
(from torch>=1.11.0->sentence-transformers) (3.1.6)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch>=1.11.0->sentence-

```

```

transformers)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparselt-cu12==0.6.2 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_cusparselt_cu12-0.6.2-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-
transformers)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-
transformers)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.11.0->sentence-
transformers)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch>=1.11.0->sentence-
transformers)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.11.0->sentence-transformers) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-
packages (from torch>=1.11.0->sentence-transformers) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from
sympy==1.13.1->torch>=1.11.0->sentence-transformers) (1.3.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-
packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers)
(2025.1.31)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn->sentence-
transformers) (3.6.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.11/dist-packages (from
astunparse>=1.6.0->tensorflow>=2.16.0->ethernet) (0.45.1)
Requirement already satisfied: google-auth<3,>=1.6.3 in
/usr/local/lib/python3.11/dist-packages (from

```

tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (2.38.0)
 Requirement already satisfied: google-auth-oauthlib<2,>=0.5 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (1.2.1)
 Requirement already satisfied: markdown>=2.6.8 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.7)
 Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (0.7.2)
 Requirement already satisfied: werkzeug>=1.0.1 in
 /usr/local/lib/python3.11/dist-packages (from
 tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr) (3.1.3)
 Requirement already satisfied: MarkupSafe>=2.0 in
 /usr/local/lib/python3.11/dist-packages (from jinja2->torch>=1.11.0->sentence-
 transformers) (3.0.2)
 Requirement already satisfied: cachetools<6.0,>=2.0.0 in
 /usr/local/lib/python3.11/dist-packages (from google-
 auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (5.5.2)
 Requirement already satisfied: pyasn1-modules>=0.2.1 in
 /usr/local/lib/python3.11/dist-packages (from google-
 auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (0.4.1)
 Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.11/dist-
 packages (from google-
 auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (4.9)
 Requirement already satisfied: requests-oauthlib>=0.7.0 in
 /usr/local/lib/python3.11/dist-packages (from google-auth-
 oauthlib<2,>=0.5->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (2.0.0)
 Requirement already satisfied: pyasn1<0.7.0,>=0.4.6 in
 /usr/local/lib/python3.11/dist-packages (from pyasn1-modules>=0.2.1->google-
 auth<3,>=1.6.3->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (0.6.1)
 Requirement already satisfied: oauthlib>=3.0.0 in
 /usr/local/lib/python3.11/dist-packages (from requests-oauthlib>=0.7.0->google-
 auth-
 oauthlib<2,>=0.5->tensorboard<2.16,>=2.15->tensorflow<2.16,>=2.7.2->ethnicolr)
 (3.2.2)
 Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl (363.4
 MB)

363.4/363.4 MB

5.0 MB/s eta 0:00:00

Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-
 manylinux2014_x86_64.whl (13.8 MB)

13.8/13.8 MB

82.2 MB/s eta 0:00:00

Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (24.6 MB)

24.6/24.6 MB

20.1 MB/s eta 0:00:00

Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)

883.7/883.7 kB

43.6 MB/s eta 0:00:00

Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)

664.8/664.8 MB

1.2 MB/s eta 0:00:00

Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl (211.5 MB)

211.5/211.5 MB

6.8 MB/s eta 0:00:00

Downloading nvidia_curand_cu12-10.3.5.147-py3-none-manylinux2014_x86_64.whl (56.3 MB)

56.3/56.3 MB

14.3 MB/s eta 0:00:00

Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-manylinux2014_x86_64.whl (127.9 MB)

127.9/127.9 MB

9.6 MB/s eta 0:00:00

Downloading nvidia_cusparsparse_cu12-12.3.1.170-py3-none-manylinux2014_x86_64.whl (207.5 MB)

207.5/207.5 MB

8.5 MB/s eta 0:00:00

Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (21.1 MB)

21.1/21.1 MB

88.0 MB/s eta 0:00:00

Installing collected packages: nvidia-nvjitlink-cu12, nvidia-curand-cu12, nvidia-cufft-cu12, nvidia-cuda-runtime-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-cu12, nvidia-cusparsparse-cu12, nvidia-cudnn-cu12, nvidia-cusolver-cu12

Attempting uninstall: nvidia-nvjitlink-cu12

Found existing installation: nvidia-nvjitlink-cu12 12.5.82

Uninstalling nvidia-nvjitlink-cu12-12.5.82:

Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82

Attempting uninstall: nvidia-curand-cu12

Found existing installation: nvidia-curand-cu12 10.3.6.82

Uninstalling nvidia-curand-cu12-10.3.6.82:

Successfully uninstalled nvidia-curand-cu12-10.3.6.82

Attempting uninstall: nvidia-cufft-cu12

Found existing installation: nvidia-cufft-cu12 11.2.3.61

Uninstalling nvidia-cufft-cu12-11.2.3.61:


```

    Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
Attempting uninstall: nvidia-cuda-runtime-cu12
    Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
    Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
        Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
Attempting uninstall: nvidia-cuda-nvrtc-cu12
    Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
    Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
        Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
Attempting uninstall: nvidia-cuda-cupti-cu12
    Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
    Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
        Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
Attempting uninstall: nvidia-cublas-cu12
    Found existing installation: nvidia-cublas-cu12 12.5.3.2
    Uninstalling nvidia-cublas-cu12-12.5.3.2:
        Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
Attempting uninstall: nvidia-cusparse-cu12
    Found existing installation: nvidia-cusparse-cu12 12.5.1.3
    Uninstalling nvidia-cusparse-cu12-12.5.1.3:
        Successfully uninstalled nvidia-cusparse-cu12-12.5.1.3
Attempting uninstall: nvidia-cudnn-cu12
    Found existing installation: nvidia-cudnn-cu12 9.3.0.75
    Uninstalling nvidia-cudnn-cu12-9.3.0.75:
        Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
Attempting uninstall: nvidia-cusolver-cu12
    Found existing installation: nvidia-cusolver-cu12 11.6.3.83
    Uninstalling nvidia-cusolver-cu12-11.6.3.83:
        Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-
cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-cu12-12.4.127
nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3 nvidia-curand-
cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-cusparse-cu12-12.3.1.170
nvidia-nvjitlink-cu12-12.4.127

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data] Package vader_lexicon is already up-to-date!

```

[34]: True

```

[35]: # Clean tweet text
def clean_text(text):
    text = re.sub(r'https?://\S+', '', text)
    text = re.sub(r'<.*?>', '', text)
    text = text.encode('ascii', 'ignore').decode('ascii')
    return re.sub(r'\s+', ' ', text).strip()

# Time & date features

```

```

def get_time_representation(timestamp):
    dt = datetime.fromtimestamp(timestamp)
    return np.array([
        np.sin(2 * np.pi * dt.minute / 60),
        np.cos(2 * np.pi * dt.minute / 60),
        np.sin(2 * np.pi * dt.hour / 24),
        np.cos(2 * np.pi * dt.hour / 24),
        np.sin(2 * np.pi * dt.weekday() / 7),
        np.cos(2 * np.pi * dt.weekday() / 7)
    ])

def get_date_representation(timestamp):
    ref_date = datetime(2015, 2, 1)
    dt = datetime.fromtimestamp(timestamp)
    return np.array([dt.toordinal() - ref_date.toordinal()])

```

```

[36]: # Device selection
device = 'cuda' if torch.cuda.is_available() else 'cpu'

# SentenceTransformer on GPU
tweet_text_model = SentenceTransformer('all-MiniLM-L6-v2', device=device)

# VADER sentiment analyzer
sentiment_analyzer = SentimentIntensityAnalyzer()

# Gender classifier on GPU
gender_classifier = pipeline(
    "text-classification",
    model="padmajabfrl/Gender-Classification",
    tokenizer="padmajabfrl/Gender-Classification",
    device=0 if torch.cuda.is_available() else -1
)

```

Device set to use cuda:0

```

[37]: # Demographic vector mapping
race_to_vector = {
    "white": np.eye(1,5,0).squeeze(),
    "black": np.eye(1,5,1).squeeze(),
    "asian": np.eye(1,5,2).squeeze(),
    "hispanic": np.eye(1,5,3).squeeze(),
    "other": np.eye(1,5,4).squeeze()
}

gender_to_vector = {
    "male": np.array([1]),
    "female": np.array([-1]),
    "unknown": np.array([0])
}

```

```

def splitname(s):
    idx = s.strip().rfind(' ')
    return ("", s) if idx == -1 else (s[:idx], s[idx+1:])

def get_race_prediction(full_name):
    df_temp = pd.DataFrame({"name": [full_name]})
    df_temp[['fname', 'lname']] = df_temp['name'].apply(lambda x: pd.
↳Series(splitname(x)))
    df_temp = ec.pred_fl_reg_name(df_temp, "fname", "lname")
    return df_temp.loc[0, "race"]

def get_gender_prediction(full_name):
    result = gender_classifier(full_name)
    return result[0]['label'].lower()

def get_demographic_embedding(full_name):
    race = get_race_prediction(full_name)
    gender = get_gender_prediction(full_name)
    race_vec = race_to_vector.get(race.lower(), race_to_vector["other"])
    gender_vec = gender_to_vector.get(gender.lower(),
↳gender_to_vector["unknown"])
    return np.concatenate([race_vec, gender_vec])

# Full feature vector creation
def get_combined_representation(tweet):
    text = tweet.get('tweet_text', '')
    text_emb = tweet_text_model.encode(clean_text(text), device=device)

    ts = float(tweet.get('timestamp', 0))
    time_emb = get_time_representation(ts)
    date_emb = get_date_representation(ts)

    name = tweet.get('user_name', '')
    demo_emb = get_demographic_embedding(name)

    sentiment = sentiment_analyzer.polarity_scores(clean_text(text))['compound']
    sentiment_emb = np.array([sentiment])

    verified_emb = np.array([1.0 if tweet.get('verified', False) else -1.0])

    return np.concatenate([text_emb, time_emb, date_emb, demo_emb,
↳sentiment_emb, verified_emb])

```

```

[38]: # # Load CSV with real impressions
# csv_path = "/content/drive/MyDrive/219_Final_project/all_tweets_flat.csv"
# df = pd.read_csv(csv_path)

```

```

# # Process rows into features + targets
# features = []
# targets = []

# for _, row in tqdm(df.iterrows(), total=len(df)):
#     tweet_dict = {
#         "tweet_text": row['text'],
#         "timestamp": row['time'],
#         "user_name": row['author'],
#         "verified": bool(row['verified'])
#     }
#     try:
#         vec = get_combined_representation(tweet_dict)
#         features.append(vec)
#         targets.append(row['impressions'])
#     except Exception as e:
#         print(f"Skipping tweet due to error: {e}")
#         continue

# X = np.vstack(features)
# y = np.array(targets)

```

```

0%|          | 214/2337437 [00:38<98:58:33,
6.56it/s]ERROR:tensorflow:=====
Object was never used (type <class
'tensorflow.python.ops.tensor_array_ops.TensorArray'>):
<tensorflow.python.ops.tensor_array_ops.TensorArray object at 0x7bd0ae080e10>
If you want to mark it as used call its "mark_used()" method.
It was originally created here:
  File "/usr/local/lib/python3.11/dist-packages/keras/src/backend.py", line
5158, in <genexpr>
    output_ta_t = tuple( File "/usr/local/lib/python3.11/dist-
packages/tensorflow/python/util/tf_should_use.py", line 288, in wrapped
    return _add_should_use_warning(fn(*args, **kwargs),
=====
0%|          | 1041/2337437 [03:06<116:06:11,  5.59it/s]

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-38-2ee42dd101ca> in <cell line: 0>()
    15     }
    16     try:
----> 17         vec = get_combined_representation(tweet_dict)
    18         features.append(vec)
    19         targets.append(row['impressions'])

```

```

<ipython-input-37-6b3b9166a6f5> in get_combined_representation(tweet)
    44
    45     name = tweet.get('user_name', '')
--> 46     demo_emb = get_demographic_embedding(name)
    47
    48     sentiment = sentiment_analyzer.
↪ polarity_scores(clean_text(text))['compound']

<ipython-input-37-6b3b9166a6f5> in get_demographic_embedding(full_name)
    29 def get_demographic_embedding(full_name):
    30     race = get_race_prediction(full_name)
--> 31     gender = get_gender_prediction(full_name)
    32     race_vec = race_to_vector.get(race.lower(), race_to_vector["other"])
    33     gender_vec = gender_to_vector.get(gender.lower(),
↪ gender_to_vector["unknown"])

<ipython-input-37-6b3b9166a6f5> in get_gender_prediction(full_name)
    24
    25 def get_gender_prediction(full_name):
--> 26     result = gender_classifier(full_name)
    27     return result[0]['label'].lower()
    28

/usr/local/lib/python3.11/dist-packages/transformers/pipelines/
↪ text_classification.py in __call__(self, inputs, **kwargs)
    157     """
    158     inputs = (inputs,)
--> 159     result = super().__call__(*inputs, **kwargs)
    160     # TODO try and retrieve it in a nicer way from
↪ _sanitize_parameters.
    161     _legacy = "top_k" not in kwargs

/usr/local/lib/python3.11/dist-packages/transformers/pipelines/base.py in
↪ __call__(self, inputs, num_workers, batch_size, *args, **kwargs)
    1360     )
    1361     else:
-> 1362         return self.run_single(inputs, preprocess_params,
↪ forward_params, postprocess_params)
    1363
    1364     def run_multi(self, inputs, preprocess_params, forward_params,
↪ postprocess_params):

/usr/local/lib/python3.11/dist-packages/transformers/pipelines/base.py in
↪ run_single(self, inputs, preprocess_params, forward_params, postprocess_params)
    1367     def run_single(self, inputs, preprocess_params, forward_params,
↪ postprocess_params):
    1368         model_inputs = self.preprocess(inputs, **preprocess_params)
-> 1369         model_outputs = self.forward(model_inputs, **forward_params)

```

```

1370         outputs = self.postprocess(model_outputs, **postprocess_params)
1371         return outputs

/usr/local/lib/python3.11/dist-packages/transformers/pipelines/base.py in
↳ forward(self, model_inputs, **forward_params)
    1267             with inference_context():
    1268                 model_inputs = self.
↳ _ensure_tensor_on_device(model_inputs, device=self.device)
-> 1269                 model_outputs = self._forward(model_inputs,
↳ **forward_params)
    1270                 model_outputs = self.
↳ _ensure_tensor_on_device(model_outputs, device=torch.device("cpu"))
    1271             else:

/usr/local/lib/python3.11/dist-packages/transformers/pipelines/
↳ text_classification.py in _forward(self, model_inputs)
    188         if "use_cache" in inspect.signature(model_forward).parameters.
↳ keys():
    189             model_inputs["use_cache"] = False
-> 190         return self.model(**model_inputs)
    191
    192     def postprocess(self, model_outputs, function_to_apply=None,
↳ top_k=1, _legacy=True):

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738     else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks
    1749         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
    1751
    1752         result = None

/usr/local/lib/python3.11/dist-packages/transformers/models/distilbert/
↳ modeling_distilbert.py in forward(self, input_ids, attention_mask, head_mask,
↳ inputs_embeds, labels, output_attentions, output_hidden_states, return_dict)
    975         return_dict = return_dict if return_dict is not None else self.
↳ config.use_return_dict
    976

```

```

--> 977         distilbert_output = self.distilbert(
    978             input_ids=input_ids,
    979             attention_mask=attention_mask,

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738         else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741         # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks
    1749         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
    1751
    1752         result = None

/usr/local/lib/python3.11/dist-packages/transformers/models/distilbert/
↳ modeling_distilbert.py in forward(self, input_ids, attention_mask, head_mask,
↳ inputs_embeds, output_attentions, output_hidden_states, return_dict)
    795         )
    796
--> 797         return self.transformer(

    798             x=embeddings,
    799             attn_mask=attention_mask,

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738         else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741         # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks
    1749         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
    1751
    1752         result = None

```

```

/usr/local/lib/python3.11/dist-packages/transformers/models/distilbert/
↳ modeling_distilbert.py in forward(self, x, attn_mask, head_mask,
↳ output_attentions, output_hidden_states, return_dict)
    548         )
    549     else:
--> 550         layer_outputs = layer_module(

    551             hidden_state,
    552             attn_mask,

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738     else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks
    1749         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
    1751
    1752     result = None

/usr/local/lib/python3.11/dist-packages/transformers/models/distilbert/
↳ modeling_distilbert.py in forward(self, x, attn_mask, head_mask,
↳ output_attentions)
    474         """
    475         # Self-Attention
--> 476         sa_output = self.attention(

    477             query=x,
    478             key=x,

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738     else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks

```



```

1749             or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
1751
1752         result = None

/usr/local/lib/python3.11/dist-packages/transformers/models/distilbert/
↳ modeling_distilbert.py in forward(self, query, key, value, mask, head_mask,
↳ output_attentions)
    388         return x.transpose(1, 2).contiguous().view(batch_size, -1,
↳ self.n_heads * dim_per_head)
    389
--> 390         q = shape(self.q_lin(query)) # (bs, n_heads, q_length,
↳ dim_per_head)
    391         k = shape(self.k_lin(key)) # (bs, n_heads, k_length,
↳ dim_per_head)
    392         v = shape(self.v_lin(value)) # (bs, n_heads, k_length,
↳ dim_per_head)

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1737         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1738     else:
-> 1739         return self._call_impl(*args, **kwargs)
    1740
    1741     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1748         or _global_backward_pre_hooks or _global_backward_hooks
    1749         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1750         return forward_call(*args, **kwargs)
    1751
    1752         result = None

/usr/local/lib/python3.11/dist-packages/torch/nn/modules/linear.py in
↳ forward(self, input)
    123
    124     def forward(self, input: Tensor) -> Tensor:
--> 125         return F.linear(input, self.weight, self.bias)
    126
    127     def extra_repr(self) -> str:

```

KeyboardInterrupt:

Dataset is far too large (2 million tweets). I cannot process this. We will randomly sample 10k tweets for a smaller dataset to train and test on.

```
[39]: # Sample 10,000 tweets
df_sampled = df.sample(n=10000, random_state=42).reset_index(drop=True)

features = []
targets = []

for _, row in tqdm(df_sampled.iterrows(), total=len(df_sampled)):
    tweet_dict = {
        "tweet_text": row['text'],
        "timestamp": row['time'],
        "user_name": row['author'],
        "verified": bool(row['verified'])
    }
    try:
        vec = get_combined_representation(tweet_dict)
        features.append(vec)
        targets.append(row['impressions'])
    except Exception as e:
        print(f"Skipped tweet due to error: {e}")
        continue

# Convert to arrays
X = np.vstack(features)
y = np.array(targets)
```

100%| | 10000/10000 [29:54<00:00, 5.57it/s]

```
[40]: # Save feature vectors (X) and targets (y)
np.savez('/content/drive/MyDrive/219_Final_project/features_targets_10k.npz',
        ↪X=X, y=y)
print("Saved features and targets!")
```

Saved features and targets!

```
[41]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        ↪random_state=42)

# Train Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Evaluate
y_pred = rf.predict(X_test)
```

```
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

MAE: 18498.841464795514

R² Score: 0.058714987097721605

```
[55]: print(f"Mean Error: {np.mean(errors):,.0f}")
      print(f"Median Error: {np.median(errors):,.0f}")
      print(f"Max Error: {np.max(errors):,.0f}")
      print(f"Min Error: {np.min(errors):,.0f}")
```

Mean Error: -4,629

Median Error: -512

Max Error: 8,240,246

Min Error: -1,705,739

```
[67]: # Compute absolute percentage error
percentage_error = np.abs((y_test - y_pred) / (y_test + 0.00000000001))

# Define correctness threshold (10% error)
threshold = 0.10 # 10%
correct_mask = percentage_error <= threshold

# Count correct predictions
num_correct = np.sum(correct_mask)
total = len(y_test)
accuracy = num_correct / total * 100

print(f"Correct Predictions (±10%): {num_correct} / {total} ({accuracy:.2f}%)")
```

Correct Predictions (±10%): 88 / 2000 (4.40%)

```
[42]: import joblib
      joblib.dump(rf, '/content/drive/MyDrive/219_Final_project/rf_model_10k.joblib')
```

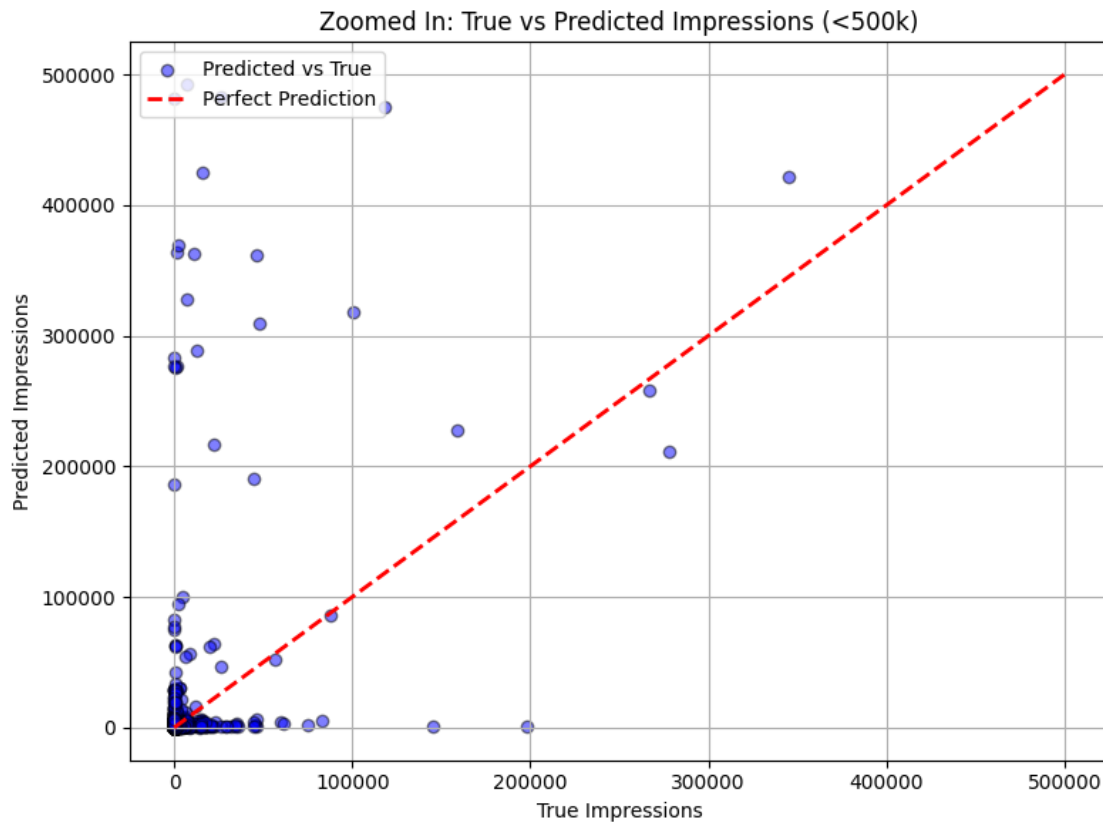
```
[42]: ['/content/drive/MyDrive/219_Final_project/rf_model_10k.joblib']
```

```
[47]: # Filter to low-range impressions
mask = (y_test < 500000) & (y_pred < 500000)

plt.figure(figsize=(8,6))
plt.scatter(y_test[mask], y_pred[mask], alpha=0.5, color='blue', edgecolor='k',
            label='Predicted vs True')
plt.plot([0, 500000], [0, 500000], 'r--', lw=2, label='Perfect Prediction')

plt.xlabel('True Impressions')
plt.ylabel('Predicted Impressions')
plt.title('Zoomed In: True vs Predicted Impressions (<500k)')
plt.grid(True)
```

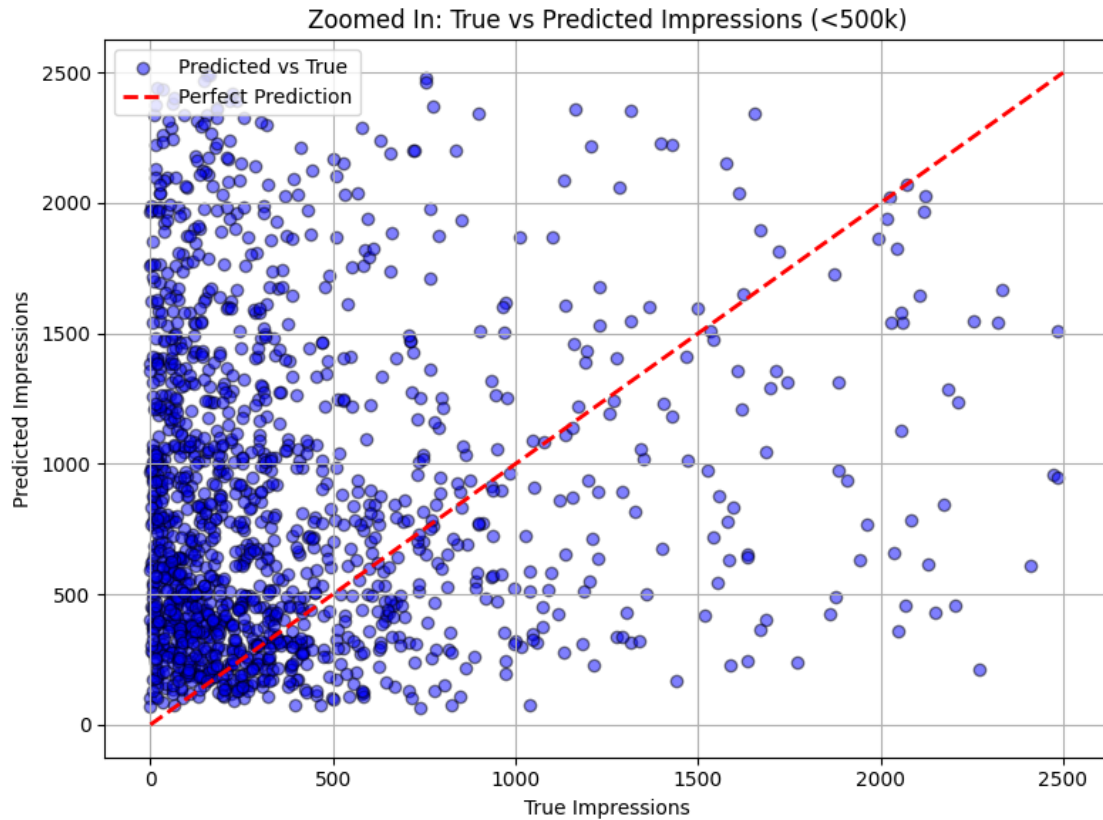
```
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```



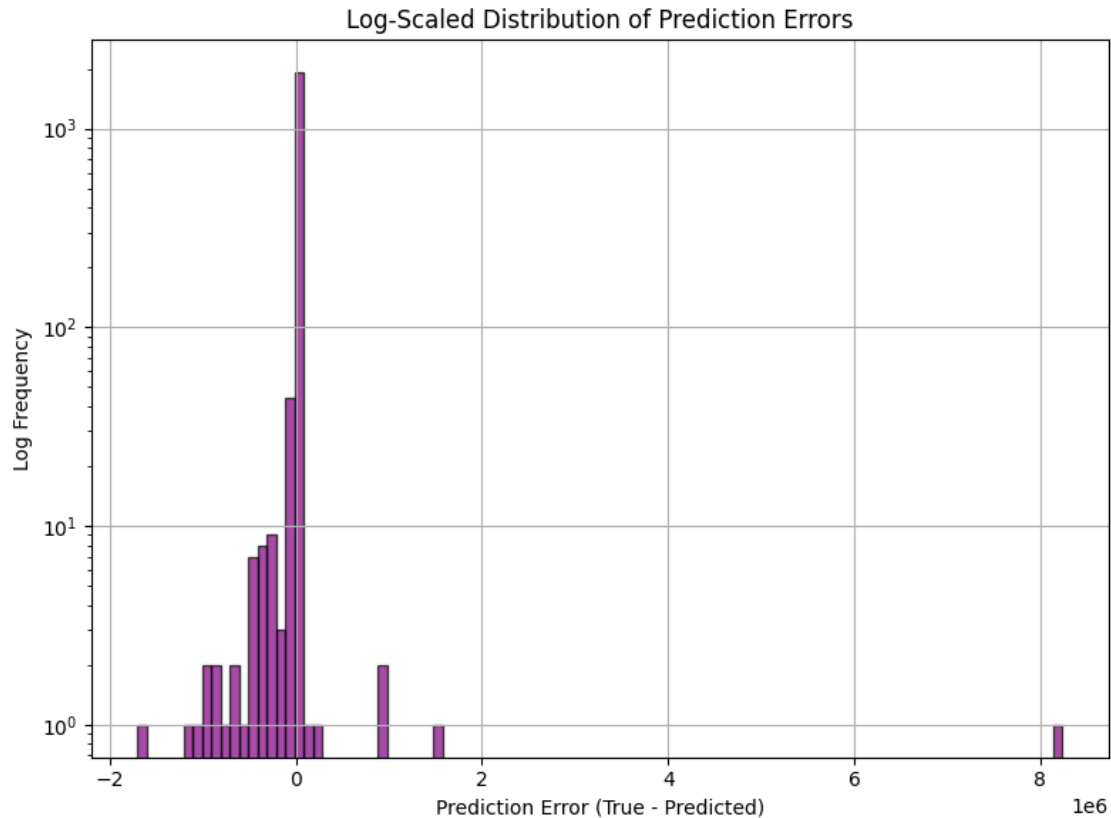
```
[54]: # Filter to low-range impressions
mask = (y_test < 2500) & (y_pred < 2500)

plt.figure(figsize=(8,6))
plt.scatter(y_test[mask], y_pred[mask], alpha=0.5, color='blue', edgecolor='k',
            label='Predicted vs True')
plt.plot([0, 2500], [0, 2500], 'r--', lw=2, label='Perfect Prediction')

plt.xlabel('True Impressions')
plt.ylabel('Predicted Impressions')
plt.title('Zoomed In: True vs Predicted Impressions (<500k)')
plt.grid(True)
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()
```



```
[66]: plt.figure(figsize=(8,6))
plt.hist(errors, bins=100, color='purple', edgecolor='black', alpha=0.7,
        log=True)
plt.xlabel('Prediction Error (True - Predicted)')
plt.ylabel('Log Frequency')
plt.title('Log-Scaled Distribution of Prediction Errors')
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
[49]: # Absolute error for each prediction
abs_errors = np.abs(errors)

# Indices of top 5 largest errors
top_indices = np.argsort(abs_errors)[-5:]

# Show True vs Predicted for worst 5
for idx in top_indices[::-1]:
    print(f"True: {y_test[idx]:.0f}, Predicted: {y_pred[idx]:.0f}, Error:␣
↪{errors[idx]:.0f}")
```

```
True: 9,181,770, Predicted: 941,524, Error: 8,240,246
True: 1,562, Predicted: 1,707,301, Error: -1,705,739
True: 1,693,181, Predicted: 122,347, Error: 1,570,834
True: 1,405, Predicted: 1,196,286, Error: -1,194,881
True: 1,522, Predicted: 1,012,007, Error: -1,010,485
```

```
[74]: from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

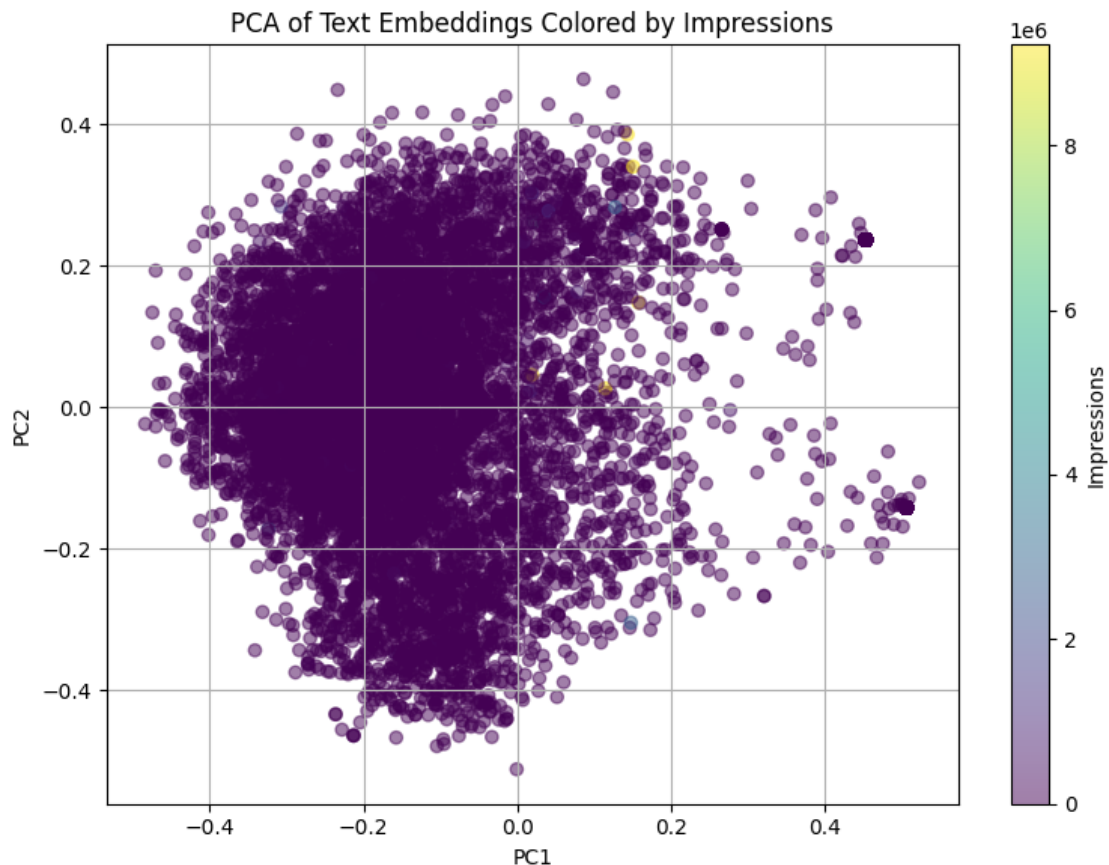
# Perform PCA on text embeddings
```

```

pca = PCA(n_components=2)
X_text_only = df_features[[col for col in df_features.columns if 'text_emb' in
                             col]].values
X_pca = pca.fit_transform(X_text_only)

# Plot PCA
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:,0], X_pca[:,1], c=df_features['impressions'],
            cmap='viridis', alpha=0.5)
plt.colorbar(label='Impressions')
plt.title('PCA of Text Embeddings Colored by Impressions')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.grid(True)
plt.tight_layout()
plt.show()

```



```

[75]: # Top 10 most important text embedding dimensions
      for i in indices:

```

```
if 'text_emb' in columns[i]:  
    print(f"{columns[i]}: {importances[i]:.4f}")
```

```
text_emb_348: 0.0556  
text_emb_195: 0.0540  
text_emb_2: 0.0462  
text_emb_24: 0.0414  
text_emb_25: 0.0376  
text_emb_68: 0.0332  
text_emb_292: 0.0276  
text_emb_154: 0.0262  
text_emb_352: 0.0252  
text_emb_286: 0.0218  
text_emb_33: 0.0185  
text_emb_79: 0.0175  
text_emb_34: 0.0164  
text_emb_339: 0.0152  
text_emb_210: 0.0140  
text_emb_285: 0.0134  
text_emb_365: 0.0121  
text_emb_117: 0.0093  
text_emb_368: 0.0087  
text_emb_376: 0.0083  
text_emb_105: 0.0080  
text_emb_282: 0.0078  
text_emb_218: 0.0072  
text_emb_139: 0.0070  
text_emb_364: 0.0065  
text_emb_246: 0.0061  
text_emb_127: 0.0059  
text_emb_312: 0.0057  
text_emb_347: 0.0056  
text_emb_369: 0.0056  
text_emb_13: 0.0055  
text_emb_54: 0.0054  
text_emb_100: 0.0052  
text_emb_326: 0.0051  
text_emb_349: 0.0049  
text_emb_60: 0.0048  
text_emb_199: 0.0046  
text_emb_130: 0.0046  
text_emb_356: 0.0045  
text_emb_329: 0.0045  
text_emb_362: 0.0044  
text_emb_51: 0.0044  
text_emb_18: 0.0043  
text_emb_371: 0.0042  
text_emb_99: 0.0041
```


text_emb_138: 0.0041
text_emb_300: 0.0041
text_emb_129: 0.0041
text_emb_178: 0.0041
text_emb_303: 0.0041
text_emb_110: 0.0040
text_emb_354: 0.0039
text_emb_264: 0.0038
text_emb_241: 0.0038
text_emb_248: 0.0038
text_emb_74: 0.0038
text_emb_207: 0.0036
text_emb_289: 0.0036
text_emb_160: 0.0036
text_emb_249: 0.0036
text_emb_3: 0.0035
text_emb_295: 0.0035
text_emb_256: 0.0034
text_emb_203: 0.0033
text_emb_235: 0.0033
text_emb_27: 0.0033
text_emb_322: 0.0033
text_emb_363: 0.0033
text_emb_280: 0.0033
text_emb_217: 0.0033
text_emb_135: 0.0032
text_emb_108: 0.0032
text_emb_167: 0.0031
text_emb_52: 0.0031
text_emb_258: 0.0031
text_emb_336: 0.0031
text_emb_149: 0.0031
text_emb_345: 0.0030
text_emb_5: 0.0030
text_emb_22: 0.0030
text_emb_220: 0.0029
text_emb_259: 0.0029
text_emb_94: 0.0029
text_emb_0: 0.0028
text_emb_317: 0.0028
text_emb_50: 0.0028
text_emb_193: 0.0027
text_emb_176: 0.0026
text_emb_41: 0.0026
text_emb_131: 0.0026
text_emb_276: 0.0025
text_emb_57: 0.0025
text_emb_174: 0.0025

text_emb_297: 0.0025
text_emb_302: 0.0024
text_emb_171: 0.0024
text_emb_263: 0.0024
text_emb_270: 0.0024
text_emb_265: 0.0023
text_emb_314: 0.0023
text_emb_21: 0.0023
text_emb_346: 0.0023
text_emb_31: 0.0023
text_emb_325: 0.0022
text_emb_49: 0.0022
text_emb_330: 0.0022
text_emb_357: 0.0022
text_emb_12: 0.0021
text_emb_361: 0.0021
text_emb_273: 0.0021
text_emb_229: 0.0020
text_emb_161: 0.0020
text_emb_316: 0.0019
text_emb_80: 0.0019
text_emb_137: 0.0019
text_emb_140: 0.0019
text_emb_283: 0.0019
text_emb_165: 0.0019
text_emb_150: 0.0018
text_emb_212: 0.0018
text_emb_128: 0.0018
text_emb_95: 0.0017
text_emb_307: 0.0017
text_emb_185: 0.0017
text_emb_279: 0.0017
text_emb_141: 0.0017
text_emb_123: 0.0016
text_emb_112: 0.0016
text_emb_274: 0.0016
text_emb_224: 0.0016
text_emb_125: 0.0016
text_emb_6: 0.0016
text_emb_247: 0.0016
text_emb_342: 0.0016
text_emb_209: 0.0015
text_emb_226: 0.0015
text_emb_230: 0.0015
text_emb_183: 0.0015
text_emb_166: 0.0015
text_emb_266: 0.0015
text_emb_187: 0.0014

text_emb_381: 0.0014
text_emb_151: 0.0014
text_emb_360: 0.0014
text_emb_155: 0.0014
text_emb_261: 0.0014
text_emb_92: 0.0013
text_emb_142: 0.0013
text_emb_275: 0.0013
text_emb_29: 0.0013
text_emb_83: 0.0012
text_emb_383: 0.0012
text_emb_298: 0.0012
text_emb_1: 0.0012
text_emb_42: 0.0012
text_emb_343: 0.0012
text_emb_308: 0.0011
text_emb_114: 0.0011
text_emb_370: 0.0011
text_emb_277: 0.0011
text_emb_158: 0.0011
text_emb_242: 0.0011
text_emb_37: 0.0010
text_emb_116: 0.0010
text_emb_291: 0.0010
text_emb_113: 0.0009
text_emb_208: 0.0009
text_emb_284: 0.0009
text_emb_81: 0.0009
text_emb_103: 0.0009
text_emb_213: 0.0008
text_emb_271: 0.0008
text_emb_47: 0.0008
text_emb_255: 0.0008
text_emb_111: 0.0008
text_emb_122: 0.0007
text_emb_366: 0.0006
text_emb_36: 0.0005
text_emb_28: 0.0005
text_emb_306: 0.0005
text_emb_215: 0.0005
text_emb_268: 0.0005
text_emb_189: 0.0005
text_emb_238: 0.0005
text_emb_157: 0.0005
text_emb_144: 0.0004
text_emb_17: 0.0004
text_emb_202: 0.0004
text_emb_77: 0.0004

text_emb_69: 0.0004
text_emb_101: 0.0004
text_emb_102: 0.0004
text_emb_121: 0.0004
text_emb_375: 0.0003
text_emb_106: 0.0003
text_emb_89: 0.0003
text_emb_43: 0.0003
text_emb_254: 0.0003
text_emb_194: 0.0003
text_emb_91: 0.0003
text_emb_35: 0.0003
text_emb_169: 0.0003
text_emb_56: 0.0003
text_emb_239: 0.0003
text_emb_46: 0.0003
text_emb_136: 0.0003
text_emb_327: 0.0002
text_emb_173: 0.0002
text_emb_62: 0.0002
text_emb_133: 0.0002
text_emb_93: 0.0002
text_emb_97: 0.0002
text_emb_10: 0.0002
text_emb_219: 0.0002
text_emb_159: 0.0002
text_emb_82: 0.0002
text_emb_351: 0.0002
text_emb_184: 0.0002
text_emb_340: 0.0002
text_emb_55: 0.0002
text_emb_243: 0.0002
text_emb_313: 0.0002
text_emb_181: 0.0002
text_emb_245: 0.0002
text_emb_8: 0.0002
text_emb_7: 0.0002
text_emb_39: 0.0002
text_emb_196: 0.0002
text_emb_253: 0.0001
text_emb_32: 0.0001
text_emb_323: 0.0001
text_emb_73: 0.0001
text_emb_88: 0.0001
text_emb_175: 0.0001
text_emb_86: 0.0001
text_emb_380: 0.0001
text_emb_58: 0.0001

text_emb_353: 0.0001
text_emb_288: 0.0001
text_emb_358: 0.0001
text_emb_304: 0.0001
text_emb_257: 0.0001
text_emb_26: 0.0001
text_emb_269: 0.0001
text_emb_204: 0.0001
text_emb_23: 0.0001
text_emb_38: 0.0001
text_emb_250: 0.0001
text_emb_221: 0.0001
text_emb_244: 0.0001
text_emb_134: 0.0001
text_emb_75: 0.0001
text_emb_287: 0.0001
text_emb_205: 0.0001
text_emb_182: 0.0001
text_emb_120: 0.0001
text_emb_162: 0.0001
text_emb_45: 0.0001
text_emb_233: 0.0001
text_emb_44: 0.0001
text_emb_310: 0.0001
text_emb_335: 0.0001
text_emb_61: 0.0001
text_emb_374: 0.0001
text_emb_318: 0.0001
text_emb_278: 0.0001
text_emb_19: 0.0001
text_emb_107: 0.0001
text_emb_67: 0.0001
text_emb_20: 0.0001
text_emb_294: 0.0001
text_emb_337: 0.0001
text_emb_355: 0.0001
text_emb_145: 0.0000
text_emb_281: 0.0000
text_emb_180: 0.0000
text_emb_48: 0.0000
text_emb_76: 0.0000
text_emb_104: 0.0000
text_emb_227: 0.0000
text_emb_223: 0.0000
text_emb_40: 0.0000
text_emb_334: 0.0000
text_emb_30: 0.0000
text_emb_118: 0.0000

text_emb_168: 0.0000
text_emb_201: 0.0000
text_emb_14: 0.0000
text_emb_9: 0.0000
text_emb_321: 0.0000
text_emb_170: 0.0000
text_emb_359: 0.0000
text_emb_126: 0.0000
text_emb_315: 0.0000
text_emb_350: 0.0000
text_emb_119: 0.0000
text_emb_87: 0.0000
text_emb_124: 0.0000
text_emb_148: 0.0000
text_emb_299: 0.0000
text_emb_84: 0.0000
text_emb_96: 0.0000
text_emb_153: 0.0000
text_emb_324: 0.0000
text_emb_338: 0.0000
text_emb_66: 0.0000
text_emb_236: 0.0000
text_emb_305: 0.0000
text_emb_90: 0.0000
text_emb_172: 0.0000
text_emb_211: 0.0000
text_emb_252: 0.0000
text_emb_379: 0.0000
text_emb_373: 0.0000
text_emb_164: 0.0000
text_emb_372: 0.0000
text_emb_232: 0.0000
text_emb_290: 0.0000
text_emb_231: 0.0000
text_emb_311: 0.0000
text_emb_72: 0.0000
text_emb_332: 0.0000
text_emb_200: 0.0000
text_emb_267: 0.0000
text_emb_115: 0.0000
text_emb_78: 0.0000
text_emb_377: 0.0000
text_emb_320: 0.0000
text_emb_109: 0.0000
text_emb_237: 0.0000
text_emb_152: 0.0000
text_emb_225: 0.0000
text_emb_146: 0.0000

text_emb_260: 0.0000
text_emb_63: 0.0000
text_emb_85: 0.0000
text_emb_11: 0.0000
text_emb_251: 0.0000
text_emb_328: 0.0000
text_emb_319: 0.0000
text_emb_234: 0.0000
text_emb_301: 0.0000
text_emb_70: 0.0000
text_emb_214: 0.0000
text_emb_132: 0.0000
text_emb_272: 0.0000
text_emb_64: 0.0000
text_emb_333: 0.0000
text_emb_293: 0.0000
text_emb_191: 0.0000
text_emb_147: 0.0000
text_emb_59: 0.0000
text_emb_177: 0.0000
text_emb_228: 0.0000
text_emb_197: 0.0000
text_emb_15: 0.0000
text_emb_188: 0.0000
text_emb_186: 0.0000
text_emb_222: 0.0000
text_emb_179: 0.0000
text_emb_98: 0.0000
text_emb_4: 0.0000
text_emb_378: 0.0000
text_emb_198: 0.0000
text_emb_309: 0.0000
text_emb_192: 0.0000
text_emb_143: 0.0000
text_emb_382: 0.0000
text_emb_71: 0.0000
text_emb_344: 0.0000
text_emb_341: 0.0000
text_emb_367: 0.0000
text_emb_296: 0.0000
text_emb_163: 0.0000
text_emb_216: 0.0000
text_emb_190: 0.0000
text_emb_240: 0.0000
text_emb_262: 0.0000
text_emb_331: 0.0000
text_emb_53: 0.0000
text_emb_65: 0.0000

```
text_emb_156: 0.0000
text_emb_16: 0.0000
text_emb_206: 0.0000
```

Goal: Understand Feature Impact Using SHAP

```
[77]: !pip install shap

import shap
import matplotlib.pyplot as plt

import shap

# Create SHAP explainer for Random Forest
explainer = shap.Explainer(rf, X_train) # Use your trained rf model

# Compute SHAP values
shap_values = explainer(X_test)

# Plot summary of feature impact
shap.summary_plot(shap_values, X_test, feature_names=columns, max_display=15)

# Choose one tweet sample
idx = 0
shap.initjs()
shap.force_plot(explainer.expected_value, shap_values[idx], X_test[idx],
    ↪ feature_names=columns)

shap.plots.bar(shap_values, max_display=15)
```

Requirement already satisfied: shap in /usr/local/lib/python3.11/dist-packages (0.46.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from shap) (1.26.4)

Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from shap) (1.14.1)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (from shap) (1.6.1)

Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from shap) (2.2.3)

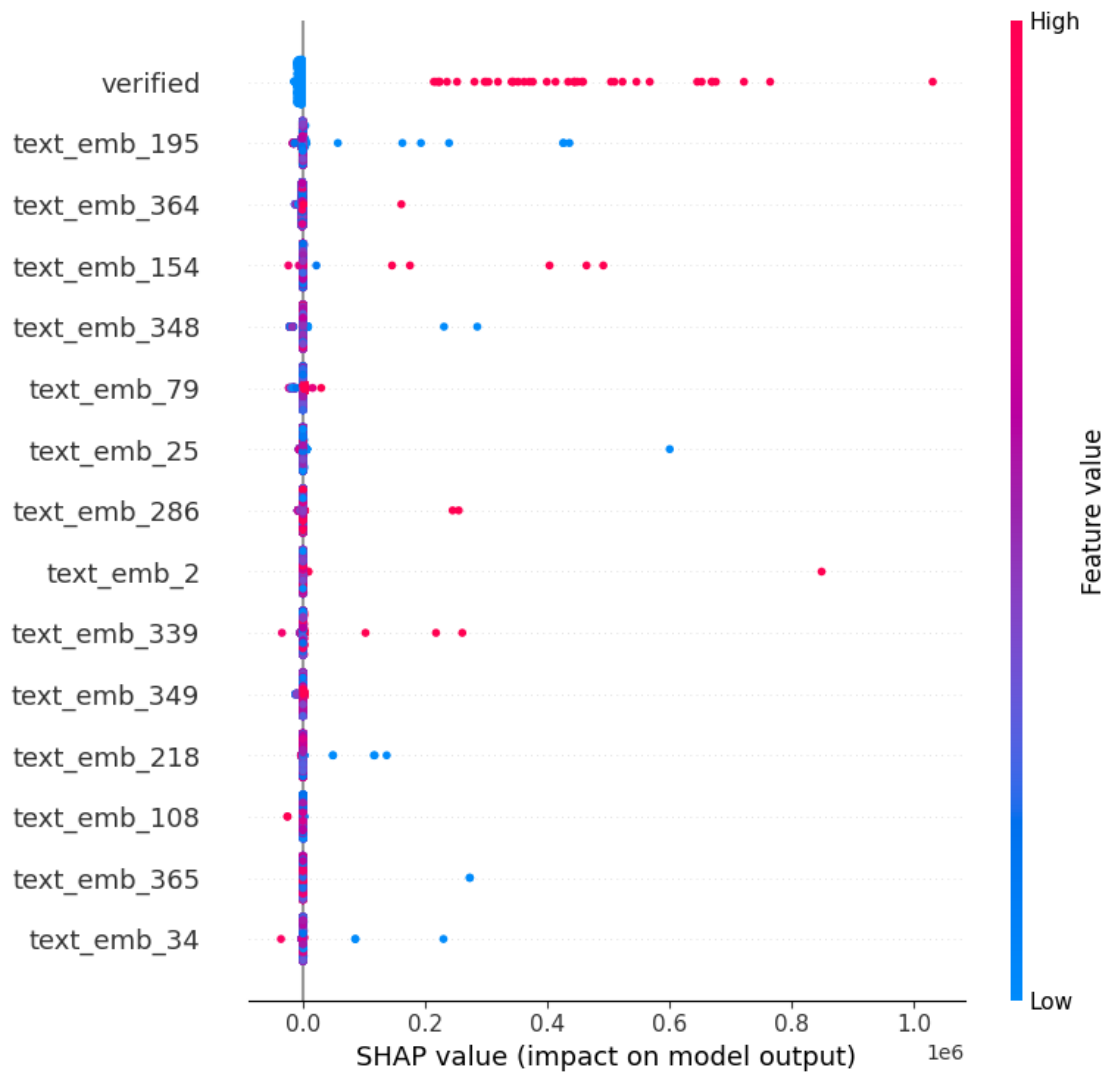
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.11/dist-packages (from shap) (4.67.1)

Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.11/dist-packages (from shap) (24.2)

Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.11/dist-packages (from shap) (0.0.8)

Requirement already satisfied: numba in /usr/local/lib/python3.11/dist-packages (from shap) (0.60.0)

Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/dist-packages (from shap) (3.1.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba->shap) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->shap) (2025.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
100%|=====| 1997/2000 [05:37<00:00]



<IPython.core.display.HTML object>

```

-----
ValueError                                Traceback (most recent call last)
<ipython-input-77-8301dd45704e> in <cell line: 0>()
    18 idx = 0
    19 shap.initjs()
----> 20 shap.force_plot(explainer.expected_value, shap_values[idx], X_test[idx],
      ↪ feature_names=columns)
    21
    22 shap.plots.bar(shap_values, max_display=15)

```

```

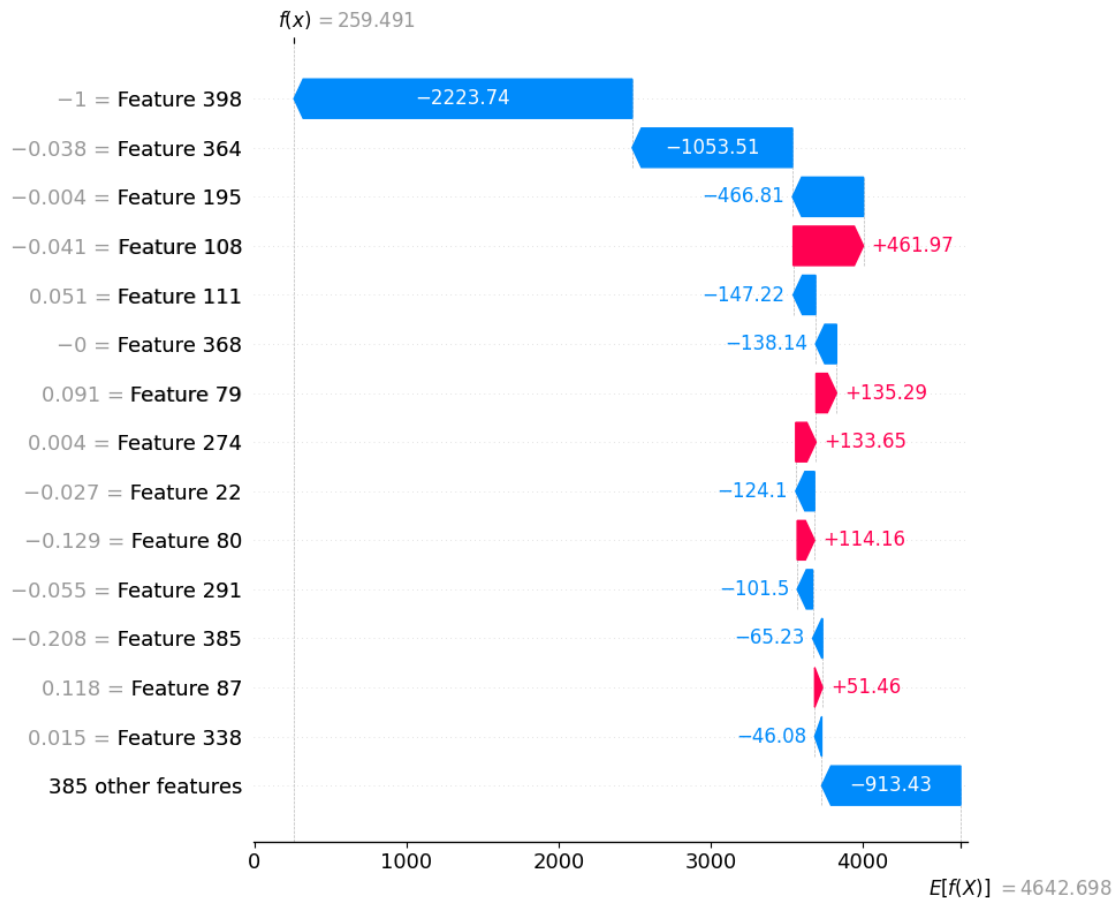
/usr/local/lib/python3.11/dist-packages/shap/plots/_force.py in
↳force(base_value, shap_values, features, feature_names, out_names, link,
↳plot_cmap, matplotlib, show, figsize, ordering_keys,
↳ordering_keys_time_format, text_rotation, contribution_threshold)
    136
    137     if type(shap_values) != np.ndarray:
--> 138         return visualize(shap_values)
    139
    140     # convert from a DataFrame or other types

/usr/local/lib/python3.11/dist-packages/shap/plots/_force.py in visualize(e,
↳plot_cmap, matplotlib, figsize, show, ordering_keys,
↳ordering_keys_time_format, text_rotation, min_perc)
    426         )
    427     else:
--> 428         raise ValueError("visualize() can only display Explanation
↳objects (or arrays of them)!")
    429
    430

```

ValueError: visualize() can only display Explanation objects (or arrays of them !

```
[78]: shap.plots.waterfall(shap_values[idx], max_display=15)
```



- $E[f(X)] = 4642.698$: The average model prediction (baseline, across all data).
- $f(x) = 259.491$: The prediction for this specific tweet.
- The model reduced the prediction from ~ 4643 to ~ 259 due to negative feature impacts.

What we have completed: - Loaded tweet data $\rightarrow$ extracted text, time, verified status, author, impressions. - Created feature vectors:

- Text embeddings (384-dim)
- Time/date, demographics, sentiment, verified (15-dim)
- Trained Random Forest $\rightarrow$ predicted impressions $\rightarrow$ Evaluated with MAE, R^2 , error plots.
- Explained model with SHAP: identified which features/text patterns drove predictions.
- Visualized insights